

Quantization for Neural Networks based on Classification Difficulty

Master thesis in the field of study "Computational Engineering" by Wanqi Yang
Date of submission: September 30, 2022

1. Review: Prof. Dr.-Ing. Li Zhang
2. Review: M.Sc. Jingcun Wang
Darmstadt



TECHNISCHE
UNIVERSITÄT
DARMSTADT

field of study:
Computational Engineering
Hardware for Artificial
Intelligence Group

Erklärung zur Abschlussarbeit gemäß § 22 Abs. 7 APB TU Darmstadt

Hiermit erkläre ich, Wanqi Yang, dass ich die vorliegende Arbeit gemäß § 22 Abs. 7 APB der TU Darmstadt selbstständig, ohne Hilfe Dritter und nur mit den angegebenen Quellen und Hilfsmitteln angefertigt habe. Ich habe mit Ausnahme der zitierten Literatur und anderer in der Arbeit genannter Quellen keine fremden Hilfsmittel benutzt. Die von mir bei der Anfertigung dieser wissenschaftlichen Arbeit wörtlich oder inhaltlich benutzte Literatur und alle anderen Quellen habe ich im Text deutlich gekennzeichnet und gesondert aufgeführt. Dies gilt auch für Quellen oder Hilfsmittel aus dem Internet.

Diese Arbeit hat in gleicher oder ähnlicher Form noch keiner Prüfungsbehörde vorgelegen.

Mir ist bekannt, dass im Falle eines Plagiats (§ 38 Abs. 2 APB) ein Täuschungsversuch vorliegt, der dazu führt, dass die Arbeit mit 5,0 bewertet und damit ein Prüfungsversuch verbraucht wird. Abschlussarbeiten dürfen nur einmal wiederholt werden.

Bei einer Thesis des Fachbereichs Architektur entspricht die eingereichte elektronische Fassung dem vorgestellten Modell und den vorgelegten Plänen.

Darmstadt, September 30, 2022

Wanqi Yang
Wanqi Yang

Contents

1	Introduction	5
1.1	Motivation	5
1.2	Objectives	7
1.3	Structure	7
2	Background	9
2.1	Fundamentals	9
2.1.1	Neural Network	9
2.1.2	Inference Acceleration	10
2.1.3	Quantization	13
2.1.4	Critical Path	18
2.2	Related State-of-the-art Work	19
3	Proposed Solution	21
3.1	Overview	21
3.2	Class-based Importance Score for Neurons and Filters	21
3.3	From Neuron to Paths	24
3.4	Class-based Sensitivity Score for Critical paths	28
3.5	Searching for the Bit-width Arrangement	30
3.6	Refining Quantized Neural Networks	33
4	Experiments and Results	35
4.1	Basic Setup of Experiments	35
4.2	A Scalable Quantization Framework	36
4.2.1	Design and Architecture	36
4.2.2	Example Workflow with ClassPathQ	38
4.3	Importance Score Experiment	40
4.4	Critical path Experiment	41
4.5	Class-path-based Quantization Experiment	46

1 Introduction

1.1 Motivation

In recent years, with advancements in neural networks and deep learning, artificial intelligence has become an essential tool in various fields, particularly in image recognition [22], object detection[42]. The scale of neural network models has grown significantly with the rapid progress of deep learning. For instance, OpenAI's GPT series has evolved from GPT-1[41] (117 million parameters) to GPT-3 [6] (175 billion parameters), demonstrating a substantial increase in model size. Similarly, the YOLO (You Only Look Once) series has expanded in both layers and complexity, from YOLOv1[43] (62 layers) to YOLOv4[4] (244 layers). Each additional layer or neuron increases the models' floating-point operations (FLOPs) and computational demands[50]. As model complexity and size grow, so do storage and deployment costs. Energy consumption also becomes a critical issue[47], as intensive computation and data transmission escalate energy usage, potentially draining mobile device batteries quickly or increasing power and cooling costs for data centers[36]. In cloud computing environments, these models must be deployed in specialized data centers or high-performance servers, presenting challenges in storage, transmission, and computation, and necessitating efforts to minimize latency and costs. Additionally, in autonomous driving[5] and general robotics, models need to be deployed on embedded systems, facing the challenge of optimizing performance within constraints like computational power and energy consumption.

To address the resource demands of models while maintaining performance, several techniques have been developed to optimize resource usage during deployment. Neural network quantization[28] is a key technique, converting high-precision floating-point values into low-bitwidth integers or fixed-point numbers. This process reduces the overall computation (FLOPs), enabling models to run on a wider range of hardware, from mid-range to low-end devices, and significantly reducing model size, which is crucial for deployment in environments with limited network bandwidth and storage. Additionally,

compressing the model size greatly lowers the energy requirements for deployment[18], ensuring more efficient and cost-effective device operation.

Quantization plays a pivotal role in advancing deep learning. Initially, many innovative applications and solutions were confined to resource-rich research labs and large corporations. However, with the advent of quantization, a broader range of companies, researchers, developers, and even hobbyists can now deploy and leverage advanced AI technologies across various hardware platforms, fostering new applications and unique research[8]. This indicates that model quantization is not merely a technical optimization but also a reflection of the demand for sustainable AI growth[45]. This approach ensures broader access to the full potential of deep learning, thereby driving further innovation and development.

Traditional quantization methods, like uniform quantization, apply the same bit width across all layers, resulting in consistent precision reduction throughout the model. While simple and predictable, this approach often compromises performance, as some parts of neural networks are more sensitive to precision changes and require higher precision. To enhance model performance, researchers have introduced mixed precision quantization[37], which assigns different precision levels across the network. However, existing methods for determining quantization precision do not consider the significance of each neuron or filter to specific classes[53]. To address this, we propose a class-path-based quantization method.

In neural networks, each neuron’s influence on the output varies by class, as someones are critical for all classes, while others significantly impact only a few. We propose that allocating bit widths based on the class-specific importance of neurons can improve model performance without compromising compression efficiency.

We introduce the concept of class paths, which are sets of neurons that significantly affect a specific class’s output. These influential neurons change the final probability of the corresponding class. Class paths are not mutually exclusive; multiple class paths can share neurons, and a single neuron can belong to multiple class paths. To identify each class’s class path, we propose a scoring method during the back-propagation of a pre-trained full-precision model, assessing each ’s influence on each class. We define the set of most influential neurons for each class as its class path based on these scores.

Using the class-path concept, we introduce class sensitivity scoring to quantify the impact of precision loss in a specific class path on other classes. Since quantizing one class path can reduce the model’s accuracy for other classes, we use this metric for evaluation.

The class-path quantization method ensures that during mixed precision quantization, bit width allocation maintains the classification accuracy for all classes rather than sacrificing some classes' accuracy to benefit others. Benchmark tests show that this method achieves results comparable to or better than current state-of-the-art methods at various compression levels.

1.2 Objectives

This study focus on exploring a neural network quantization approach based on classification difficulty, which is the class path quantization.

Inspired by the critical path, we built the quantization method for channel quantization. This is achieved by identifying the model's critical path for each class, which we name class path and investigating the interconnections amongst class paths of different classes.

The fundamental heuristic is such that if a critical path of class is significantly influenced by the critical paths of other classes, then the neurons on this path should be assigned a higher number of quantization bits. This is essential to maintain its accuracy and thereby assure reliable classification outcomes. Conversely, if a class's critical path is less affected by the critical paths of other classes, the neurons on this critical path could opt for fewer quantization bits without causing substantial degradation in classification accuracy.

1.3 Structure

This thesis consists of the following chapters:

Chapter 2 provides basic information for understanding the technical aspects of the thesis.

In addition, the context of the research problem and a comparison of the current work with previous work are given.

Chapter 3 describes the proposed solution for the research problem, which is a quantization strategy.

Chapter 4 describes the process and results of the experiments for the proposed quantitative strategy.

Chapter 5 summarizes the findings and discusses future work.

2 Background

This chapter will introduce the basic terminology and methods used in this thesis. Instead of giving specific examples, this chapter will provide an introduction. Understanding these basics will help the reader understand the technical content associated with this thesis. In addition, we will investigate the relevant literature, and introduce the relationship between them and our study.

2.1 Fundamentals

2.1.1 Neural Network

A neural network consists of an input layer, an intermediate or hidden layer, and an output layer, while a deep neural network (DNN) generally refers to a neural network with multiple hidden layers, where each layer of the neural network can be considered as an extraction of different types of features from the input data. Neural networks can be divided into three main categories according to their structure: MultiLayer Perceptron (MLP)[3], Convolutional Neural Network (CNN)[32] and Recurrent Neural Network (RNN)[24].

In a multilayer perceptron (MLP), neurons between layers are primarily connected by fully connected (FC) layers. These FC layers abstract the structure of biological neurons mathematically, processing multiple external inputs using basic tensor dot product operations and transmitting the results to subsequent layers. Convolutional neural networks (CNNs) contain numerous convolutional (Conv) layers to effectively handle the spatial information in two-dimensional images. In a Conv layer, the convolution kernel multiplies and sums the matrix elements corresponding to the receptive field on the feature map, then moves to the next position using a sliding window. This method, which mirrors the human visual system, significantly reduces network parameters through weight sharing.

CNNs are widely used in computer vision tasks such as image classification. Recurrent neural networks (RNNs) often employ long short-term memory (LSTM) units to retain information from previous inputs. These networks excel at processing sequential data with contextual relationships, such as text and speech, making them suitable for tasks like speech recognition and language translation. LSTM units involve multiple vector-matrix multiplications and use gating mechanisms to control the flow of information[17].

After each layer in the networks completes its computations, an activation function is applied. The main purpose of this function is to simulate the behavior of biological neurons, determining whether a neuron should be activated or inhibited based on whether the input reaches a certain threshold. Mathematically, activation functions introduce non-linearity into neural network models, enhancing their generalization capabilities. Common activation functions include Sigmoid, Tanh[34], and ReLU[15]. ReLU, the most frequently used rectified linear unit, simplifies the computation process and effectively prevents issues such as gradient explosion and gradient vanishing.

Neural network training employs the backpropagation algorithm[44], which consists of two main processes: forward propagation and backward propagation. During forward propagation, input samples pass through the input layer, intermediate hidden layers, and finally reach the output layer. In supervised learning, the network's output is compared to the target output to calculate the loss function. To minimize this error, gradients of the loss function with respect to the network parameters are computed. Using the chain rule, these gradients propagate backward through the network. Stochastic gradient descent is then typically used to update the network parameters, improving the accuracy of subsequent predictions. This process is repeated until the model converges to an optimal solution[21].

2.1.2 Inference Acceleration

Accelerating neural network inference requires interdisciplinary collaboration, involving machine learning, optimization, computer architecture, data compression, and hardware design. This area has attracted significant attention from the deep learning community and has seen considerable advancements in recent years. These methods are generally classified into four categories: parameter pruning and sharing, low-rank decomposition, compact convolution filters, and knowledge distillation.

Parameter Pruning and Sharing

The reduction of redundancy, whether it pertains to informational redundancy or model parameter redundancy, can primarily be achieved through three methods: model quantization, parameter sharing, and structural matrices. Model quantization will be elaborated upon in subsequent chapters.

The practices of network pruning and parameter sharing have been employed to alleviate network complexity and mitigate overfitting [16]. An early incarnation of pruning methods is the Biased Weight Decay [19]. Within this realm, the Optimal Brain Damage [33] and Optimal Brain Surgeon [20] techniques aimed to reduce connections based on the Hessian matrix of the loss function. Comparative studies have indicated that the precision of these pruning methods surpasses those based on importance, such as the weight decay method. However, there exist inherent challenges with pruning and sharing methodologies. First, when L1 or L2 regularization is utilized, pruning methods necessitate a higher number of iterations for convergence. In addition, all pruning techniques mandate manual tuning for layer sensitivity, requiring meticulous hyperparameter adjustments, which can be challenging in certain applications.

Structured matrices are also an interesting approach. A matrix with order $m \times n$ is a structured matrix if it requires less than $m \times n$ parameters to describe [46]. Generally, such a structure not only reduces storage consumption, but also significantly speeds up inference and training through fast matrix-vector multiplication and gradient computation.

Low-rank Decomposition and Sparsity

A typical CNN convolution kernel is a 4D tensor, and it is important to note that there can be a lot of redundancy in these tensors. The idea based on tensor decomposition may be a promising way to reduce redundancy [10]. The fully connected layer can be treated as a 2D matrix, and a low-rank decomposition is also feasible [9].

All approximations are done layer by layer, and after a layer has been approximated by a low-rank filter, the parameters of that layer are fixed, while previous layers have been fine-tuned using a reconstruction error criterion. This is a typical low-rank approach to compressing 2D convolutional layers.

Low-rank methods are well suited for model compression and acceleration, and the methods complement recent developments in deep learning such as dropout, rectified unit, and maxout. However, the implementation of low-rank methods is not easy as it involves

computationally expensive decomposition operations. First, current methods perform the low-rank approximation layer by layer and cannot perform the significant global parameter compression because different layers have different information. Additionally, the decomposition requires a lot of retraining to reach convergence.

Knowledge Distillation

To our understanding, the pioneering work in leveraging knowledge transfer (KT) for model compression was introduced by Caruana et al [7]. They employed pseudo-data annotated by an ensemble of strong classifiers to train a compressed model, which subsequently replicated the outputs of the original, larger network. However, their research was mainly focused on shallow networks. This foundational concept was later expanded upon, renamed as "Knowledge Distillation" (KD) [2]. This method enabled the compression of deeper and broader networks into more compact architectures, with the latter emulating the capabilities of its intricate counterpart. At its core, KD strives to distill the vast knowledge from a large "teacher" model into a more compact version by learning the class distributions from the teacher's softened softmax outputs.

A subsequent study embedded KD within a compression framework that minimized the training overhead of deep networks by adhering to the student-teacher paradigm [23]. This paradigm imparts knowledge by penalizing the student's deviations from the softened outputs of the teacher. This strategy effectively compresses the deep "teacher" network into a student network of equivalent depth. The ultimate goal is for the student to be adept at predicting the outputs of the teacher, which are the authentic classification labels. Despite its apparent simplicity, the KD approach has consistently delivered commendable performance across a myriad of image classification tasks.

While KD-centric techniques have the potential to render deeper models more concise, substantially reducing computational overhead, they are not devoid of limitations. For instance, KD methods are mainly tailored for classification tasks utilizing the softmax loss function, constraining their applicability. Moreover, the assumptions underpinning certain KD models can sometimes be overly strict, causing their performance to occasionally lag behind alternative strategies.

2.1.3 Quantization

In most deep learning frameworks, the parameters and computations of the default model are stored and represented in 32-bit floating-point type. After quantizing the parameters of the neural network model, the quantized low-precision parameters can effectively reduce the size of the parameters and the memory consumption, and reduce the computational difficulty, which is an effective optimization scheme to accelerate the forward inference of the model[14]. The fundamental idea of model quantization is to replace the original floating-point precision with a low-precision data type that is less costly to store and compute, and to make a compromise between accuracy and the cost of storage and speed.

Uniform and Non-uniform Quantization

The mapping between the values in a linear relationship is called uniform quantization, and the general basic transformation from floating-point to fixed-point type can be written as the equation

$$Q = \text{round}(\frac{R}{S} - Z) \quad (2.1)$$

$$\text{round}(x) = [x] \quad (2.2)$$

Among them, $\text{round}(\cdot)$ refers to the rounding function, as shown in Equation 2.2, which is often realized by the ceil function in programming languages, Q is the fixed-point value after quantization, R is the floating-point value before quantization, S is the smallest scale that can be expressed after quantization, and Z is the value corresponding to the zero in the original value domain after quantization, and Z denotes a zero offset, which is due to the fact that the parameter of the neural network often contains a zero value, and it is necessary to allow the zero of the original value domain to get the accurate representation after quantization.

Correspondingly, the fixed-to-floating-point inverse quantization formula can be written as

$$R = (Q + Z) \times S \quad (2.3)$$

Commonly, in order to make the quantization map to an integer range of a specified bit width (e.g., n bits), the linear transformation factor S can be calculated by the following equation

$$S = \frac{R_{max} - R_{min}}{2^n - 1} \quad (2.4)$$

where R_{max} and R_{min} denote the maximum and minimum floating-point values of the original value domain, respectively, and n denotes the quantization target bit width.

When the numerical distribution of the original value domain, there are more very large and very small outliers. From the point of view of information entropy considerations, the above S calculation will waste unnecessary bit width in the information-intensive region, resulting in the loss of information. Therefore, consider in the calculation of the transformation factor S statistical numerical distribution, the first part of the less informative anomalies for the truncation operation, the floating-point type is converted to a fixed-point quantization formula can be written as

$$Q = \text{round}\left(\frac{\text{clip}(R, \alpha, \beta)}{S} - Z\right) \quad (2.5)$$

$$\text{clip}(x, \alpha, \beta) = \begin{cases} \alpha, & x \geq \alpha \\ x, & \alpha < x < \beta \\ \beta, & x \leq \beta \end{cases} \quad (2.6)$$

where $\text{clip}(\cdot)$ is the truncation function and α and β are the upper and lower bounds of the truncation interval, respectively, called the truncation parameters in the quantization algorithm.

In general, the distribution of parameters in a convolutional neural network is non-uniform, and the region often has some intervals with denser points and some with sparser points. From the point of view of information transfer, more bandwidth can be allocated to the intervals with dense points, reducing the bandwidth of sparse intervals in a way that reduces the overall loss of the quantization information. As shown in Figure 2.1, the corresponding way is non-uniform quantization, which refers to unequal lengths between quantization levels, such as using logarithmic functions to establish mapping relationships, or considering larger degrees of freedom and more complex mapping methods, in order to retain the information of dense interval points. Non-uniform quantization can better

ensure the accuracy of the quantized model [56][25], but it is not conducive to hardware computation for optimization and acceleration, and requires specific hardware devices [1].

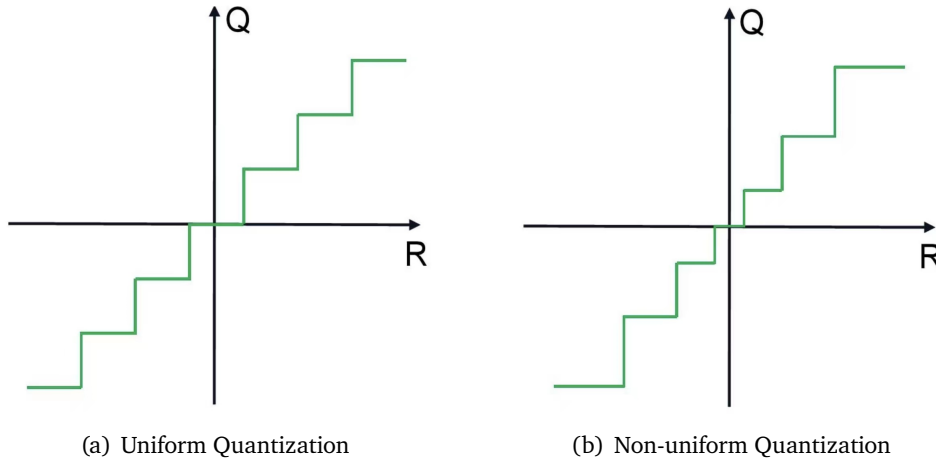


Figure 2.1: Uniform and Non-uniform Quantization

Symmetric and Asymmetric Quantization

As shown in the Figure 2.2, in the quantization process, the value domain before and after quantization is required to be symmetric about the zero point, i.e., when the zero offset Z is 0, it is called symmetric quantization, on the contrary, when the zero offset Z is not equal to 0, it is called asymmetric quantization.

From the point of view of information coding, asymmetric quantization has the advantage of being able to make full use of the bits compared to asymmetric quantization, since symmetric quantization results in an increased loss of information as the degree of uneven distribution increases. From the perspective of computational complexity, symmetric quantization is less computationally difficult than asymmetric quantization. In the parameters of neural network model, the input feature map and the model weight parameter distribution have their own characteristics. When the parameter distribution is close to the symmetry of about 0 (such as the Gaussian distribution), symmetric quantization is suitable, while like the feature values due to the characteristics of the activation func-

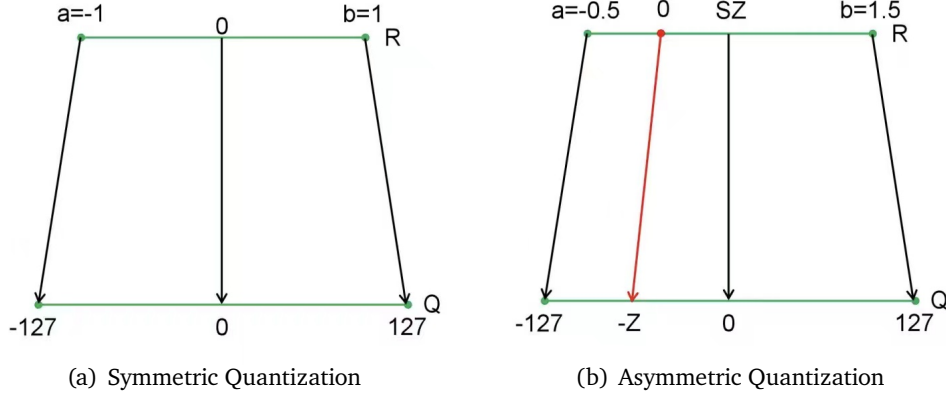


Figure 2.2: Symmetric and asymmetric quantization

tion (such as ReLU, etc.), often non-negative, theoretically more suitable for asymmetric quantization.

Different Quantization Granularity

The main computational unit of the convolutional neural network consists of convolutions, these convolutional units make up each layer of the network. There are a large number of convolutional filters in the network, and each convolutional filter can have their own different weight parameters. The range of their weight parameter values influences the quantization deflation factor S in the calculation of the clipping range $[\alpha, \beta]$ of the granularity. The approaches of the network quantization can be classified into the three types: layer-wise quantization, group-wise quantization, channel-wise quantization.

The clipping range parameter of the layer-wise quantization is determined by calculating the weights of all convolutional kernels parameters in the entire layer [30]. As shown in the Figure 2.3, the statistical information of all the weight parameters of a layer (such as maximum, minimum values, percentage of value range) is evaluated, and then the same clipping range parameter is adopted for the whole layer of convolutional filters. It is easier to compute and to implement. However, when the range of convolutional weight parameters in the layer varies a lot, this kind of method makes the convolutional kernel

with a relatively small parameter range lose its quantization resolution and lose more information.

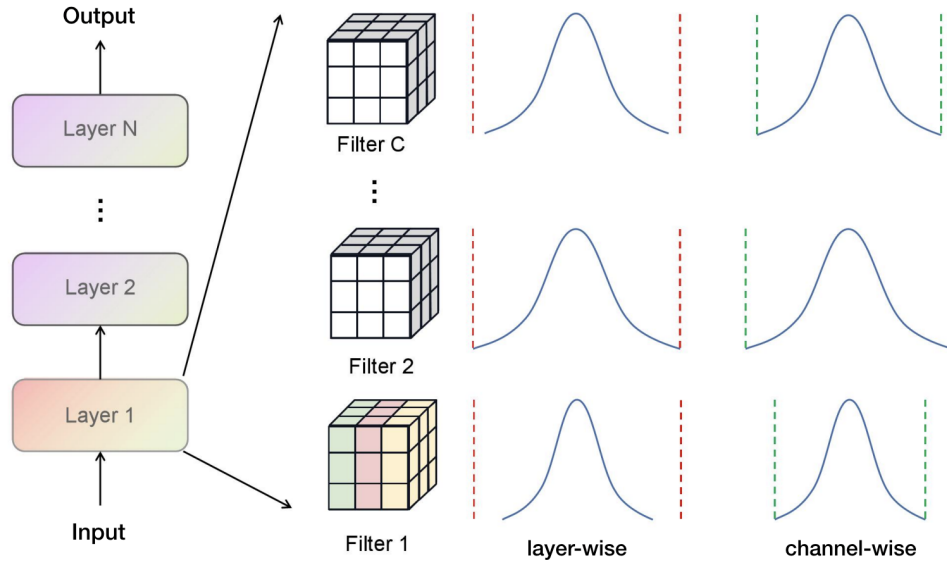


Figure 2.3: Layer-wise and channel-wise quantization

Group-wise quantization is similar to the group-wise convolution. With the group as the basic unit, using the same quantization parameter to calculate the clipping range among a group, which consists of multiple channels within a single layer. This is a solution to the inability of layer quantization methods to handle the distributions of the convolution range parameter varying a lot in the same layer. The group-wise quantization is a compromise between quantization resolution and computational complexity.

Channel-wise quantization is a special case of making full use of group-wise quantization. A dedicated quantization parameter is computed for each convolutional kernel, and independent of the other channels [26][28]. This ensures better quantization resolution, preserves more of the original information, and generally achieves better accuracy compared to the previous two methods.

2.1.4 Critical Path

In neural networks, the influence of each neuron or filter on the final output varies. As these networks grow in complexity, certain neurons' output changes during forward propagation become significant to the final output. This set of neurons is referred to as the "Critical Path."

Many methods aim to identify these critical paths, such as NeuronIntGrad [49], Neuron-MCT [29], DGR [52], and Greedy Pruning. These methods differ in techniques, often focusing on the contributions of individual neurons and the specific objectives of their identification process.

Research on the critical path partially relies on neural theory, which posits that each neuron in a neural network contributes differently to the output, thereby identifying neurons of varying importance. Additionally, studies have indicated that in classification tasks, the contribution of individual neurons differs across classification outcomes. Figure 2.4 illustrates the results of an experiment within the study. The network is a multilayer perceptron (MLP) designed to classify images of cats and dogs. Notably, the neurons influencing the classification of cats and dogs are distinct. Some neurons significantly affect only one class during forward propagation, while others impact both classes. The neuron at the far right of the first layer does not contribute to either class.

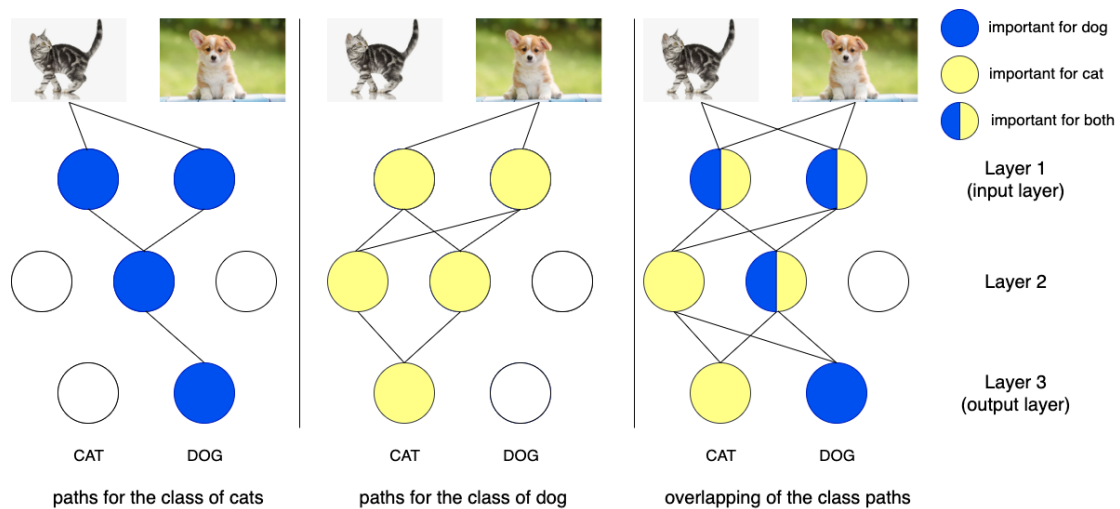


Figure 2.4: An example of the data path and the importance of neurons for different classes.

Thus, we can identify crucial neurons for each class in output and combine all significant neurons for a particular class into a set, which can be termed the class’s critical path i.e. “class path.”

We primarily use the NeuronMCT method, which assesses neuron importance by calculating activation gradients. Its validated effectiveness in identifying key neurons and class paths makes it our preferred approach.

2.2 Related State-of-the-art Work

Many neural network quantization strategies have been proposed to achieve model compression and inference acceleration without significant loss of model accuracy.

[27] focuses on enhancing the training process for model-level uniform quantized networks. By using gradient scaling, it reduces back-propagation errors. On the other hand, [38] targets performance improvement for model-level uniform quantized networks, especially those run on low bit-width accumulators. It achieves this by tweaking the loss function. [54] brings in an innovative approach by utilizing multiple batch normalization settings. This allows model-level quantized neural networks to adjust their quantization bit-width post-training. To increase inference accuracy, it also employs knowledge distillation. Meanwhile, [35] taps into the power of knowledge distillation to refine the outcomes of networks that have undergone model-level uniform quantization.

However, there’s a common limitation across these methods: they overlook the adaptability offered by multi-bit quantization. This technique provides variable bit-widths to layers or channels according to their importance. Key layers or channels get higher bit-widths, while the less critical ones get lower. It optimally shrinks neural network sizes without sacrificing inference accuracy. The real challenge lies in determining the right bit-width for different network sections.

To address this, [51] employs reinforcement learning to set bit-width at the layer level. But, this layer-level approach does not get as detailed as filter-level quantization. Due to the vast search space, using reinforcement learning for filter-level bit-width determination is tricky. To counter this, [55] devises a loss-based iterative technique for filter-level bit-width settings. However, it leans more towards non-uniform quantization, requiring several back propagation iterations to pinpoint the optimal bit-width for individual filters.

The identical model structure may be used for diverse classification tasks. This brings attention to the notion of task-specific quantization methods, which have shown the potential to optimize neural networks. [48] pioneered this approach. They employed the NeuronMCT method to ascertain the importance score of each neuron for individual classes. The summation of these importance scores across all classes for a neuron was then utilized as a heuristic. The objective was to optimize the average bit count of quantization. This approach was guided by a fundamental principle: neurons with greater significance receive a higher quantization bit allocation.

However, this method of assigning quantization bit-widths overlooks the interrelationships between neurons during quantization. If neurons in deeper layers are assigned fewer bits, it may limit the information processing potential of neurons in shallower layers.

A potential solution draws on the critical path theory. [39] highlights the "path specialization" phenomenon, noting that different classes activate distinct neural network segments during inference. Expanding on this, [13] applied the critical path concept for adversarial detection, enhancing inference robustness. They found a correlation between class path similarity and classification task complexity: the more similar the critical paths of two classes, the harder they are to distinguish.

Based on the relationship between the critical path and the classification task, we proposed a quantization method which is based on classification difficulty.

3 Proposed Solution

3.1 Overview

In this chapter, we detail our proposed quantization approach. We begin by setting a target bit-width for the neural network and then reduce the average bit-width accordingly. For quantizing a pre-trained full-precision model, we first perform a single backpropagation to determine the importance score of each neuron/filter for each class. We then select the neurons/filters with the highest importance scores for each class, defining these as the critical path for that class. Next, we calculate sensitivity scores for each class-specific critical path, indicating the influence each path has on others. Using these sensitivity scores, a search algorithm determines the optimal bit-width for each neuron/filter, enabling model quantization with this bit-width configuration. Finally, we refine the model to restore accuracy.

In Section 3.1, we explain how to calculate the importance scores of neurons and filters. Section 3.2 covers building the critical path for each class. Section 3.3 details the calculation of sensitivity scores for critical paths across all classes. In Section 3.4, we present the search algorithm for determining bit arrangements using critical path information, and Section 3.5 introduces the refining process.

3.2 Class-based Importance Score for Neurons and Filters

To measure the importance of a neuron or filter for the neural network’s output, we employ a class-based method, assigning each neuron/filter an importance score for each class. This score helps us allocate higher bit-widths to neurons that are crucial for multiple classes. The importance score for each neuron is calculated, and the score for each filter is determined by selecting the maximum value among its related neurons.

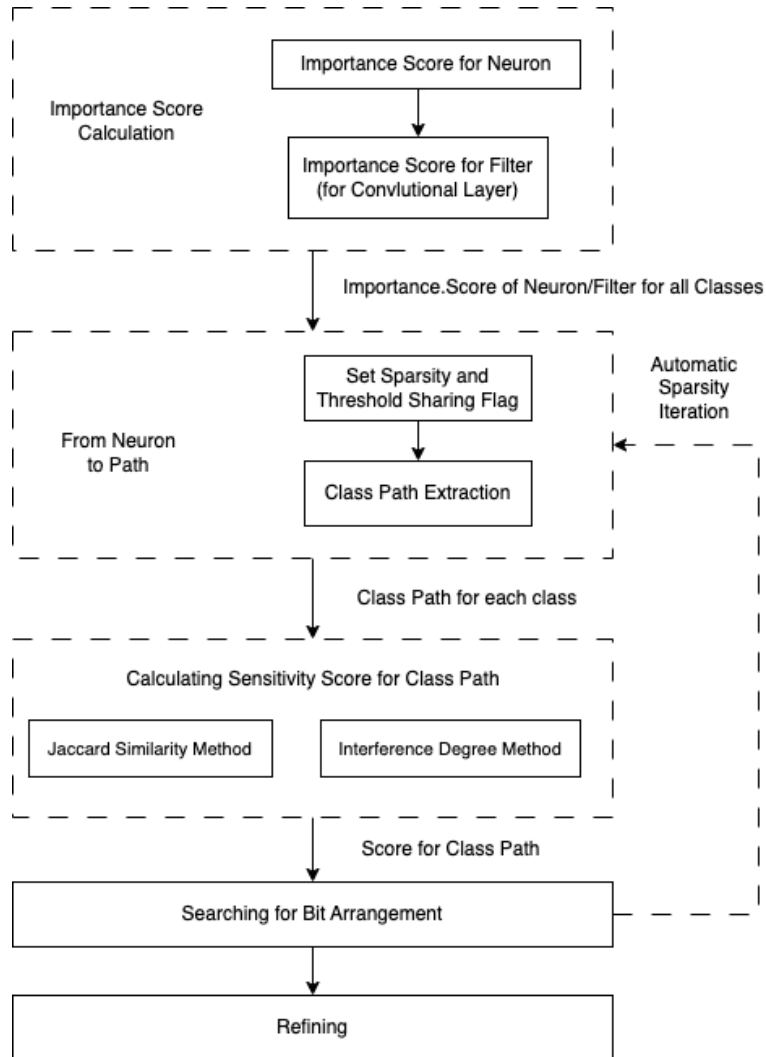


Figure 3.1: An Overview of Pipeline for Proposed Solution

Critical path theory posits that neurons contribute differently to a model's output, and the critical path is formed by the most important neurons. This theory also provides methods for calculating each neuron's importance score for each class. The calculation starts by determining how much the model's output changes when a neuron is removed. Thus, a neuron's importance score is measured by the difference in the model's output with and without that neuron. For an input image x_m , $m \in \{1, \dots, M\}$ is the index of class, M is the number of the classes. The effect of removing a neuron is called the marginal contribution and we define it as the importance score of a neuron with respect to image x_m . So for the neuron $j \in \{1, \dots, N_i\}$ in the layer $i \in \{1, \dots, L\}$ for a single image x_m , L is the number of all layers, the importance score $s_{(i,j)}^m$ can be written as

$$s_{(i,j)}^m = \left| \Phi_{\theta}(x_m) - \Phi_{\theta}(x_m; a_j^i \leftarrow 0) \right| \quad (3.1)$$

where $\Phi_{\theta}(x_m)$ denotes the output of the neural network for sample x_m and a_j^i is the activation of neuron j in the layer i . $a_j^i \leftarrow 0$ that the activation of the neuron j in the layer i is frozen at zero. But computing the exact value for the marginal contribution of all neurons is computationally expensive. As we have to ablate each neuron (total N) in the network and observe its effect after inference. Therefore we use a Taylor approximation,

$$s_{(i,j)}^m = \left| a_j^i \nabla_{a_j^i} \Phi_{\theta}(x_m) \right| \quad (3.2)$$

where $\nabla_{a_j^i}$ is the gradient of the output of the model with respect to the mask of the neuron j in the layer i . In this way, we only need to perform the back propagation once to obtain the importance scores of all the neurons for image x_m . Once we've determined the significance scores for each neuron, we utilize a threshold, ϵ , to ascertain if the neurons are part of the critical path. If the score $s_{(i,j)}^m$ surpasses ϵ , then neuron j in layer i is identified as a part of the critical paths for image x_m . In practice, ϵ ought to be a value nearly zero. In our study, we assigned ϵ a value of 10^{-5} . Subsequently, a batch of validation images from class m , sized N_s , are fed to the model. With enough big batch size N_s , we can get the importance score $\beta_{(i,j)}^m$ of neuron j in the layer i for class m from the set s_j^i including the scores from all images in the batch with the equation

$$\beta_{(i,j)}^m = \frac{1}{N_s} \left| \left\{ s \in s_j^i \mid s > \epsilon \right\} \right| \quad (3.3)$$

s is the importance score of a single data for a neuron j in the layer i .

Since each filter contains multiple neurons, each with its own importance score for each class, setting different bit-widths for neurons within a single filter is impractical for several reasons. First, it increases storage overhead, as varying bit-widths for each weight require more storage space, potentially enlarging the model size. Second, it complicates processor design, necessitating decoding of each weight’s bit-width and communication with each multiplier. Lastly, while lower bit-widths may speed up individual multipliers in vector multiplication, overall computation is limited by the slowest multiplier, hindering performance. Therefore, we calculate a single importance score for each filter by using the maximum score of all related neurons. So we define the importance score $\varphi_{i,k}^m$ of a filter k in $\{1, \dots, C_i\}$, C_i is the number of filter in layer i with equation

$$\varphi_{(i,k)}^m = \max \left\{ \gamma \mid \gamma \in \Gamma_{(i,k)}^m \right\} \quad (3.4)$$

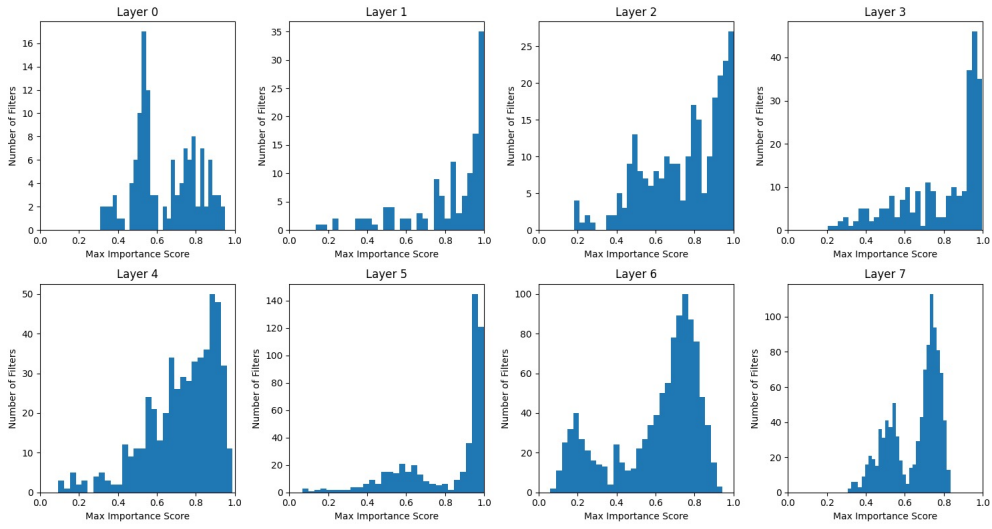
where $\Gamma_{(i,k)}^m$ is the set of importance scores for class m defined in Equation (3.3), and γ is the importance score of a neuron in $\Gamma_{(i,k)}^m$. By the above method, the importance scoring of neurons and filters is calculated for the following.

3.3 From Neuron to Paths

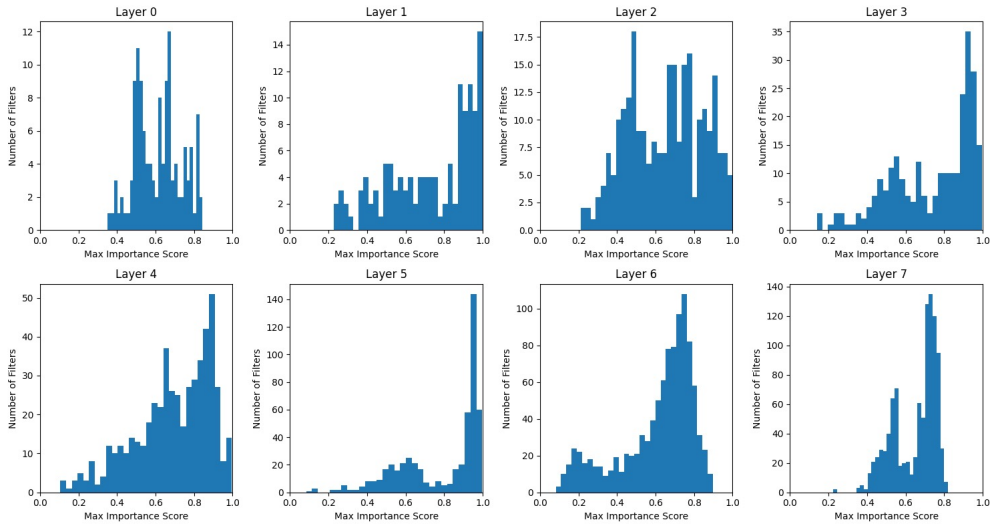
Importance scoring reflects each neuron’s significance for each class, allowing us to calculate the class path by identifying the most important neurons. First, we need to establish two presets. Given that our importance scores range from 0 to 1, the first preset is whether to use a uniform threshold of importance scores for all classes. The second preset determines the proportion of neurons to select as critical paths, distinguishing important from unimportant neurons for each class.

Based on these two presets, the following path extraction process is conducted. First, the importance scores for each class are sorted: neuron importance scores for fully connected layers and filter importance scores for convolutional layers. Next, the threshold is determined. This step needs to be discussed based on two pre-settings divided into two cases:

- If the threshold sharing between classes is set, then all classes share the same threshold. According to the sparsity α setting, we first merge and sort the importance scores of all classes, select the top α scores, and refer to these scores as being the important scores of their corresponding classes, and their corresponding neurons as



(a) importance score distribution for class 0



(b) importance score distribution for class 5

Figure 3.2: Histograms of the number of filters versus the importance scores for class 0 and 5 in a floating-point VGG-small network trained on CIFAR10. The x-axis shows the number of filters, and the y-axis shows the importance scores of filter

the important neurons of the corresponding classes. The important neurons of each class constitute the critical path of each class.

- If it is set that thresholds are not shared between classes, then each class computes its own threshold. According to the setting of sparsity α , we first sort the importance scores of each class separately, and for the sorted scores of each class, we select the top α scores respectively, and refer to these scores as being the important scores of their corresponding classes, and their corresponding neurons as the important neurons of the corresponding classes. That is, each class maintains a threshold value of its own, which is the smallest of the importance scores of the important neurons of that class.

If a neuron does not belong to any of the critical paths, then we call it an unimportant neuron. The set of unimportant neurons participates as a path in the later bit search, but not in the scoring phase, where we manually set the post-scoring sorting position for it.

An example as shown in Figure 3.3 is given to explain this process. For example, in a classification problem that distinguishes two classes A, B, if there are four neurons whose importance scores for A are, respectively, 0.9, 0.6, 0.7, and 0.1, and whose importance scores for B are, respectively, 0.4, 0.3, 0.7, and 0.5, and assuming that the sparsity is 50%, then, in the presence of shared thresholds, we need to first merge all the scores, and then select the the top 50% scores, which are 0.6, 0.7, 0.7, 0.9, while neurons 0, 1, and 2 form the critical path for class A and neuron 2 form the critical path for class B, neuron 3 form the critical path for unimportant neurons. In the case where each class has a separate threshold, we need to just sort each class individually and then select the important neurons according to the sparsity of 50%, neurons 0, 2 form the critical path of class A, and neurons 2, 3 form the critical path of class B.

When thresholds are shared across classes, a uniform criterion determines neuron importance for multiple classes, leading to different critical path sizes for each class. Intuitively, easier-to-categorize classes may need fewer important neurons, while harder-to-categorize classes need more.

However, in our sensitivity scoring approach, a class with a small critical path but significant interference with other paths may receive lower search priority due to its size, resulting in fewer allocated bits and reduced accuracy. By maintaining individual thresholds for each class, the critical paths remain the same size, ensuring an equal number of important neurons per class and avoiding this issue.

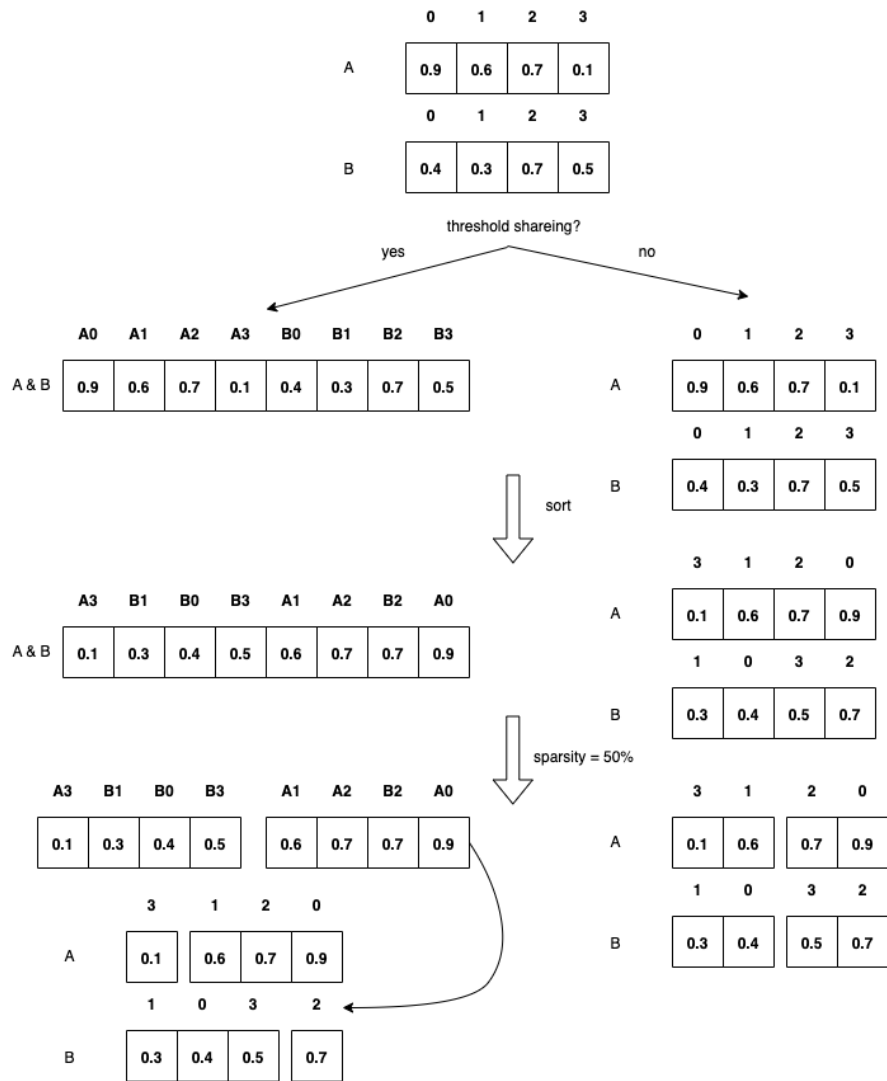


Figure 3.3: An example about how the threshold and sparsity effect the path extraction

3.4 Class-based Sensitivity Score for Critical paths

After extracting the paths, we need to evaluate the sensitivity of each path, assessing how accuracy loss on one path impacts other paths, i.e., how sensitive the model output is to the accuracy of each path.

We propose two options for evaluating the impact of one critical path on another:

Option 1: Jaccard Similarity. Jaccard Similarity measures the similarities and differences between sets of samples. A higher Jaccard Similarity indicates greater similarity. We use it to calculate the similarity between the critical paths of any two classes. The equation for sensitivity score is

$$S_J(A, B) = J(C_A, C_B) = \frac{|C_A \cap C_B|}{|C_A \cup C_B|} \quad (3.5)$$

where $S_J(A, B)$ is that, the sensitivity score with Jaccard Similarity method for class A and B . And $J(C_A, C_B)$ is Jaccard Similarity of critical paths C_A and C_B for class A and B .

Option 2: Interference Degree. We define the interference degree of source class A on target class B as the sum of the importance scores of all overlapping neurons of the critical paths of source class A and the critical paths of target class B on target class B . The equation for sensitivity score is

$$S_I(A, B) = \sum_{e \in C_A \cap C_B} I_B(e) \quad (3.6)$$

where $S_I(A, B)$ is that, the sensitivity score with Interference Degree method for source class A and target class B , and e is neuron from $C_A \cap C_B$, $I_B(e)$ is importance score of the neuron e for target class B .

An example about these 2 methods are as shown in Figure 3.4. This example includes the input information for calculation of sensitivity score. For network with 7 neuron, each neuron has its own importance score for class cat and dog. The critical path for class cat includes neuron A,B,C,D,G and for class dog includes neuron A,B,C,D,F. So the overlapping part of these 2 class paths is neuron A,B,C,D. So the Jaccard Similarity of these 2 classes is

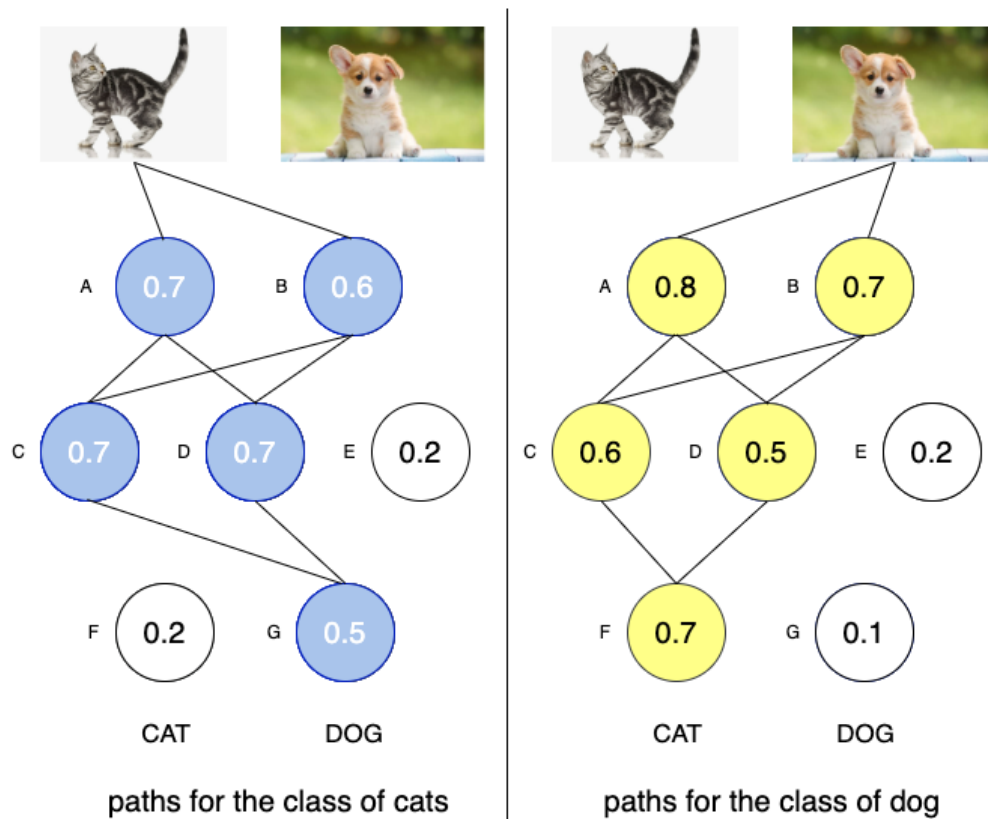


Figure 3.4: An example for showing how sensitivity score is calculated. In each network, every neuron has its own importance score for cat or dog, and the neuron with color is important neuron

$$S_J(Cat, Dog) = J(C_{cat}, C_{dog}) = \frac{|A, B, C, D|}{|A, B, C, D, F, G|} = 2/3 \quad (3.7)$$

The Interference Degree from cat to dog is

$$S_I(Cat, Dog) = \sum_{e \in C_{cat} \cap C_{dog}} I_{dog}(e) = 0.8 + 0.7 + 0.6 + 0.5 = 0.26 \quad (3.8)$$

and the Interference Degree from dog to cat is

$$S_I(Dog, Cat) = \sum_{e \in C_{cat} \cap C_{dog}} I_{cat}(e) = 0.7 + 0.6 + 0.7 + 0.7 = 0.27 \quad (3.9)$$

After completing the sensitivity scoring for each class pair, we summed the sensitivity scores for each class. For the Jaccard Similarity method, we added all Jaccard Similarities involving that class. For the Interference Degree method, we summed all Interference Degrees where that class is the source.

3.5 Searching for the Bit-width Arrangement

Classes are sorted based on the sensitivity scores of their critical paths, with unimportant neurons inserted into the ranking according to the sparsity setting. Higher sensitivity scores indicate that quantizing with fewer bits could affect more classes and result in greater accuracy loss. Therefore, in the bit search, critical paths with higher sensitivity scores are allocated more bits, minimizing accuracy loss, while paths with lower sensitivity scores are allocated fewer bits, accepting greater precision loss.

We propose an intuition-based strategy for placing unimportant neurons based on sparsity:

- **High sparsity:** Place unimportant neurons at the front of the ranking.
- **Medium sparsity:** Place unimportant neurons in the middle of the ranking.
- **Low sparsity:** Place unimportant neurons at the end of the ranking.

Before starting the bit search, we address overlapping neurons or filters between critical paths. When a neuron belongs to multiple critical paths, it should adopt the highest bit setting among them. For example, as shown in Figure 3.5, if neuron A is in two critical paths, one set to 3 bits and the other to 4 bits, it will be set to 4 bits. Similarly, in experiments involving critical path deletion and retention, a neuron present in both a deleted and a retained path will follow the highest bit setting and be retained.

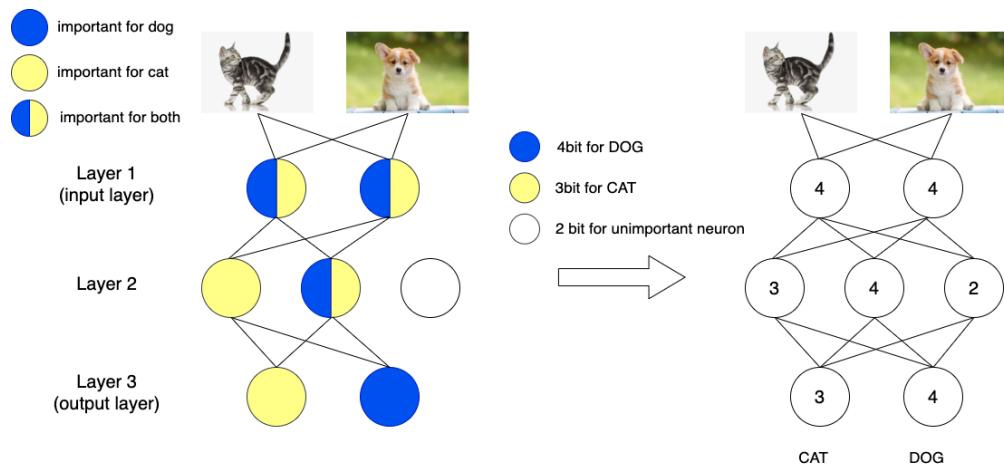


Figure 3.5: A Example to show how to set bit for neuron when it belongs more than 1 critical path

The next step is to find the appropriate bit allocation that meets the target bit width while maintaining accuracy. The algorithm adjusts bit widths assigned to different critical path classes in the model, ideally allocating more bits to higher-sensitivity classes.

We introduce an advanced mixed-precision quantization strategy. Neurons within each critical path are quantized to the same number of bits, ensuring consistent precision representation. For overlapping neurons between multiple critical paths, we use the highest bit setting as described earlier. This approach stabilizes and refines the quantization process, maintaining the integrity and efficiency of critical paths. Additionally, it leverages classification difficulty information associated with critical paths, enhancing model accuracy and robustness.

In classification tasks with limited categories, manual bit allocation based on category order can ensure appropriate representation. However, in complex multi-class tasks, manual allocation becomes impractical. Therefore, we developed an innovative bit search algorithm to address this challenge.

The algorithm works as follows: we first assign a uniform bit value N to the critical paths of all categories, where N represents the maximum number of bits that can be accepted in this bit allocation task. When we set this initial value, the inference accuracy of the model on the test set is A_{top} , which represents the highest accuracy of the current model. Next, based on the sensitivity scores of each class, we begin a stepwise reduction of the bits for the critical paths of the class. Starting with the critical path with the lowest sensitivity score, we reduce its bits one fixed step D at a time from the current B_0 . It is worth noting that as the bits are progressively reduced, the inference accuracy of the model decreases accordingly.

However, a key observation is that a change in the critical path of a particular class does not just affect the class itself, but may have a knock-on effect on all classes throughout the model. Therefore, to ensure that the overall performance of the model is within acceptable limits, we use the model's inference accuracy over the entire test set as our search control variable. Specifically, when the accuracy of the model decreases to A_0 , we will stop reducing the bits further. Here, A_0 is obtained by a specific formula:

$$A_0 = T_0 * (A_{top} - A_{baseline}) + A_{baseline} \quad (3.10)$$

where $A_{baseline}$ represents a baseline accuracy, which is defined manually, and T_0 is a predefined scaling factor that determines the magnitude of accuracy degradation we are willing to accept.

After completing the bit-width search for the first category, we arrange each category in ascending order of sensitivity. For each specific category, the search commences from its current bit-width, denoted as B_m , and decrementally proceeds in a consistent step size. Herein, $m \in \{1, \dots, M\}$ signifies the index of the category under consideration, whilst M represents the total number of categories.

As the bit-width adjustment transpires, should the model's accuracy experience a decrement surpassing the predetermined threshold A_m for the corresponding category, we desist from further adjustments for that category and transition to the subsequent one. The computation of A_m adheres to the formula:

$$A_m = R \times (A_{top} - A_{baseline}) + A_{baseline} \quad (3.11)$$

where R acts as a discount factor intended to modulate the magnitude of accuracy's decrement. Simultaneously, $A_{baseline}$ stands for the a priori established baseline accuracy.

If, during this iterative process, the average bit-width across all categories falls below the target bit-width, the search procedure is terminated. The current bit-width configuration is then adopted as the final outcome. This approach ensures that while adjusting the bit-width, the overall model performance is not unduly compromised.

In cases where the target average bit-width is stringent, the search algorithm may not always meet the set objectives. To address this, we proposed an auxiliary optimization strategy.

We systematically reduce the bit-width of each class’s critical path, prioritizing classes by ascending sensitivity. Starting with the class of least sensitivity, we decrement its bit-width in steps. Once the minimal permissible bit-width, typically one bit, is reached, we reassess the model’s average bit-width against the target.

If the average bit-width still does not meet the target after minimizing the bit-width of the least sensitive class, we apply the process to the next class with higher sensitivity. This iterative approach continues until the model’s average bit-width aligns with the target.

3.6 Refining Quantized Neural Networks

The quantization process, though efficient, might compromise model accuracy. To address this, we introduce a refining step to restore performance using knowledge distillation. This technique leverages insights from the full-precision model (the teacher) to guide and refine the quantized model (the student).

To express this mathematically, we define the loss function, L_{kd} , in the refining phase as:

$$L_{kd} = \alpha * L_{ce} + (1 - \alpha) \sum_{k=1}^M Y_k \log\left(\frac{Y_k^{fc}}{Y_k}\right) \quad (3.12)$$

In the aforementioned equation, α is a temperature factor ranging from 0 to 1, balancing soft and hard labels. L_{ce} represents the cross-entropy loss of the original neural network. The term $\sum_{k=1}^M Y_k \log\left(\frac{Y_k^{fc}}{Y_k}\right)$ represents the Kullback-Leibler divergence, where M is the total number of classes, and Y_k^{fc} and Y_k are the k-th outputs from the full-precision and quantized networks, respectively.

In training quantized neural networks with knowledge distillation, determining the gradient for the quantized weights poses a challenge. A common solution is using the straight-through estimator (STE) during backpropagation. In our experiments, we integrated STE in the refinement phase of quantized neural network training. Preliminary results show that this inclusion significantly enhances the network’s overall accuracy.

4 Experiments and Results

4.1 Basic Setup of Experiments

In order to show the effect of the class path quantization, we employed three distinct neural network configurations. First, the VGG-small, which was taken from the study [40], stands as a simplified version of the traditional VGG networks. It retains the essence of the VGG architecture but with a reduced number of layers and parameters, making it easy for applications requiring faster computations without significant compromises on accuracy. The architecture of the VGG-small model is summarized in Table 4.1.

Next, we use ResNet-20, an architecture from [22]. ResNet, short for Residual Network, features a deep but manageable structure using skip connections to bypass certain layers, enabling efficient training of deeper networks without the vanishing gradient problem. In our experiments, we used two variants of ResNet-20: expand-1 (ResNet-20-x1) and expand-5 (ResNet-20-x5). The "expand" term indicates the scaling factor for the network's width, allowing us to study its behavior with different complexities. Table 4.2 illustrates the architecture of the ResNet-20x1.

For the empirical evaluation, we used two widely recognized datasets: CIFAR10 and CIFAR100 from [31]. CIFAR10 contains 60,000 32x32 color images across 10 classes, with 6,000 images per class, split into 50,000 training and 10,000 testing images. CIFAR100, similarly structured, includes 100 classes, presenting a finer categorization and a more challenging classification task.

We implemented the algorithm and neural networks using PyTorch, a deep learning framework that provides dynamic computation graphs for flexible model design. Our experiments were conducted on the Nvidia A100, a high-performance GPU designed for deep learning computations.

4.2 A Scalable Quantization Framework

To explore the relationship between class scoring, class paths, and quantization through extensive experimentation, we developed a reusable and scalable per-channel quantization framework. This framework abstracts the processes of scoring, path extraction, quantization, and other configurations into independent, loosely coupled components. Consequently, it allows flexible adjustment of each component via scripts, enabling the generation of a vast array of experimental results.

As a modular quantization framework, it empowers developers to customize various aspects of neural network quantization. The key features include:

- Construction of models using the PyTorch library with model-level quantization metrics.
- Integration of operators specifically designed for network quantization, with defined operator-level quantization metrics.
- Development of custom trainers that seamlessly integrate training and fine-tuning into the model workflow.
- A unified configuration interface for network quantization and a consistent trainer interface for both operators and models, simplifying the definition and integration of quantizers.
- Support for per-channel quantization across all currently implemented operators.

4.2.1 Design and Architecture

To implement a robust quantization framework, it is essential to support quantization at various levels. The structure of our framework is depicted in Figure 4.1.

At the core of quantization lies the fundamental function of transforming floating-point tensors into fixed-point tensors. This transformation is guided by quantization parameters such as bit-width, maximum value, and minimum value. Leveraging this core function, we develop both activation quantizers and weight quantizers. Typically, weights are fixed post-training, allowing for either layer-wise or channel-wise quantization. Channel-wise quantization, in particular, has become the standard practice due to hardware support. On the other hand, since activation values change dynamically during inference, they are

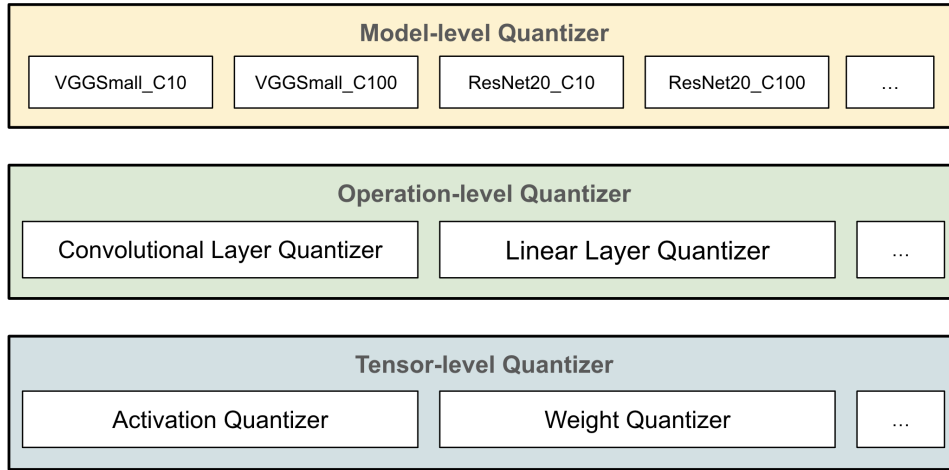


Figure 4.1: Different levels of quantizers

generally quantized on a layer-wise basis. Users can specify bit-width for each weight channel to enable channel-wise quantization, and similarly, set bit-width for each activation layer to enable layer-wise quantization.

For operation-level quantization, we define a base class, `OpQuant`, which provides a unified interface for calculating multiply-accumulate operations (MACs) after quantization. Each specific quantization operation class inherits from both `OpQuant` and the corresponding PyTorch operation, utilizing the tensor-level quantization interface to override the forward propagation process.

For instance, in a quantized 2D convolutional layer, the forward propagation is divided into two steps. First, based on the quantization configuration, the floating-point values of the input and weights are transformed into fixed-point values. Then, in the second step, these fixed-point values are processed by the PyTorch convolution operator, as illustrated in Figure 4.2

Using operator-level quantizers that inherit from their corresponding PyTorch operations, we can construct a model-level quantizer. To facilitate this, we define a base class, `ModelQuant`, which provides a unified interface for setting quantization configurations and calculating the multiply-accumulate operations (MACs) of a quantized model, leveraging

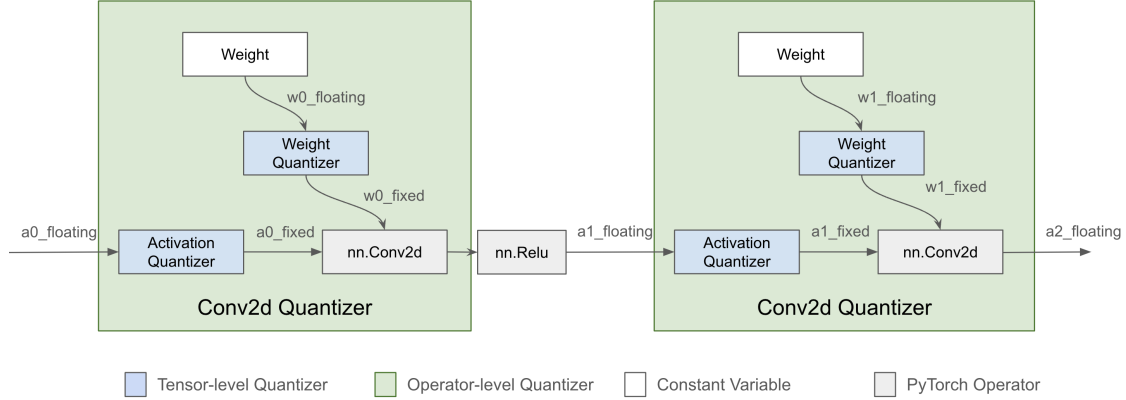


Figure 4.2: Example of quantized 2d convolutional layer

the related interfaces of operation-level quantization. Building a quantized model is as straightforward as constructing a model in PyTorch: simply replace the PyTorch operations with their corresponding quantizers.

In addition to constructing model-level quantizers, we abstract processes such as training and quantization into independent elements. The Trainer base class defines two essential interfaces: one for loading configurations and another for training, both of which are necessary to encapsulate the entire training process. Based on this, we developed SimpleTrainer for training floating-point models and KDTrainer for fine-tuning quantized models using knowledge distillation.

In a quantization framework, the quantizer component demands the highest degree of flexibility. To keep it as straightforward as possible, we defined minimal interfaces for loading configurations and processing quantization. Following this design, we developed ClassPathQuantizer and ClassNeuronQuantizer.

4.2.2 Example Workflow with ClassPathQ

To illustrate how the quantization framework works, we choose our proposed ClassPathQ as example to show the workflow.

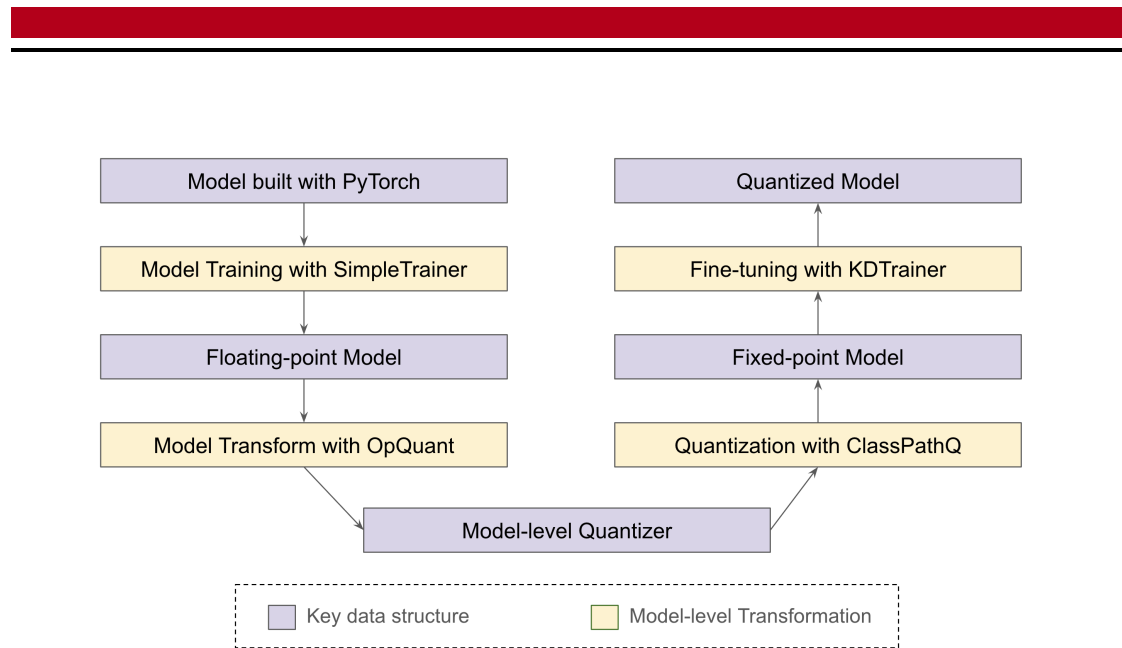


Figure 4.3: Example of quantization workflow

The entire process of the quantization with ClassPathQ is illustrated in Figure 4.3. The workflow begins by using the PyTorch API to build a neural network model, such as VGGSmall. The next step involves configuring the SimpleTrainer to train this model, resulting in a floating-point model with strong performance on the classification task.

Following this, the operations in the trained model are replaced with our operation-level quantizers—a process we refer to as model transformation. This yields a model that is prepared for quantization, allowing us to set the bit-width for the model. After configuring and executing the ClassPathQuantizer, a fixed-point model is produced. However, the performance of this initial quantized model is typically suboptimal.

To address this, the final step in the workflow involves fine-tuning the fixed-point model using KDTrainer. This step is crucial as it helps the quantized model recover its performance, bringing it closer to that of the original floating-point model.

4.3 Importance Score Experiment

Based on the experimental setup, we first calculated the importance scores for each experimental combination and saved them as intermediate outputs. The importance scores differ dimensionally between the convolutional and fully connected layers: for convolutional layers, we keep a set of importance scores for each filter, and for fully connected layers, for each neuron. Each set contains one importance score per class.

Using VGG-small + CIFAR10 as an example, the distribution of scores for each class is very similar, such as for classes 0 and 5 shown in Figure 3.2. The histograms for other classes are also similar to these. However, summing the scores of each neuron or filter across all classes revealed that the total scores in each layer follow a normal distribution, as shown in Figure 4.4. This difference indicates that some neurons score high for certain classes and low for others, confirming that significant neurons contribute differently to different classes.

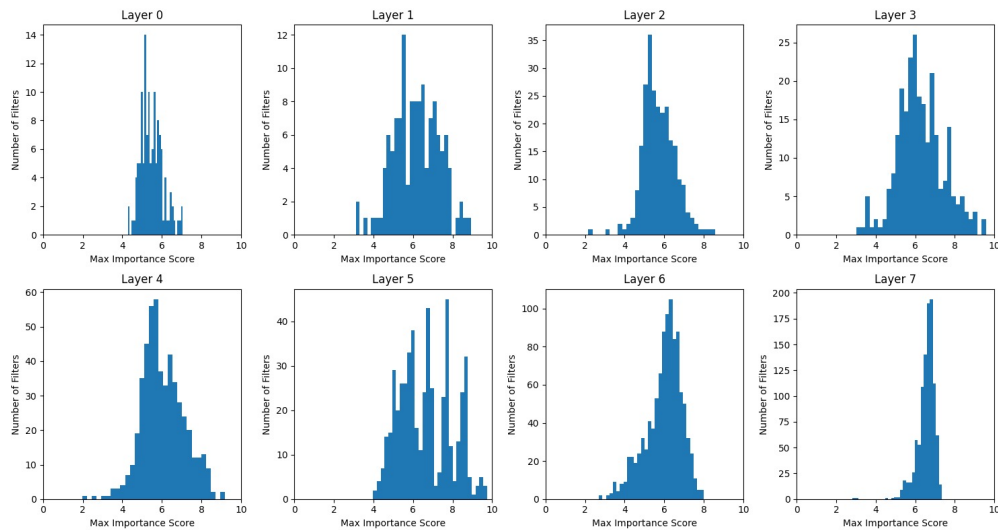


Figure 4.4: Histograms of the number of filters versus the sum of importance scores for all class in a floating-point VGG-small network trained on CIFAR10. The x-axis shows the number of filters, and the y-axis shows the importance scores of filter

4.4 Critical path Experiment

After calculating the importance scores, we used the results for critical path extraction. We conducted several experiments on the critical paths of the extracted classes, including removing only the critical paths of one class to observe how the inference results are affected.

To investigate the specific impact of critical paths on classification results, we first removed the critical paths of individual classes and observed the changes. We set the sparsity to 0.5 in the VGG-small + CIFAR10 experiment with a non-sharing threshold. In the obtained critical paths, we set the activations of individual classes' critical paths to 0 sequentially.

As Table 4.3 shows, removing any single critical path did not significantly impact the inference results for its corresponding class. Typically, deleting the critical path of one class led to a decrease in accuracy across all classes, with no clear pattern. However, we also found that setting the critical paths of non-significant neurons to 0 had a larger impact on the inference results for all classes. Based on this observation, in later experiments with higher sparsity, we prioritized unimportant neurons in the ordering of classes.

In order to further investigate the relationship between critical paths and classification, we first took a closer look at the classification task of a full-precision VGG-small model on the CIFAR10 dataset. Figure 4.5(a) is the confusion matrix of a VGG-small model, which trained on the CIFAR10 training set, on its test set. And the Figure 4.6 shows the Jaccard Similarity matrix for the critical paths of each class under setting $sparsity = 0.6$. And in Figure 4.5(b), the Jaccard Similarity matrix of ResNet-20-x1 model for the critical paths of each class under setting $sparsity = 0.7$ is shown.

Class confusion occurs when the model misclassifies one class as another. For example, confusing cats and dogs means the model incorrectly identifies cat images as dogs and dog images as cats.

Comparing the Jaccard Similarity matrices of the VGG-small and ResNet-20-x1 models, we find that the similarity matrices are essentially the same. High class path similarity in VGG-small corresponds to high similarity in ResNet-20-x1, and low similarity in VGG-small corresponds to low similarity in ResNet-20-x1. This suggests that path overlap is related to classification difficulty rather than model structure.

The CIFAR10 dataset includes classes such as airplane, car, bird, cat, deer, dog, frog, horse, boat, and truck. The Jaccard Similarity matrix shows higher similarity for the critical paths of horse and deer, and cat and dog, indicating these pairs are harder to

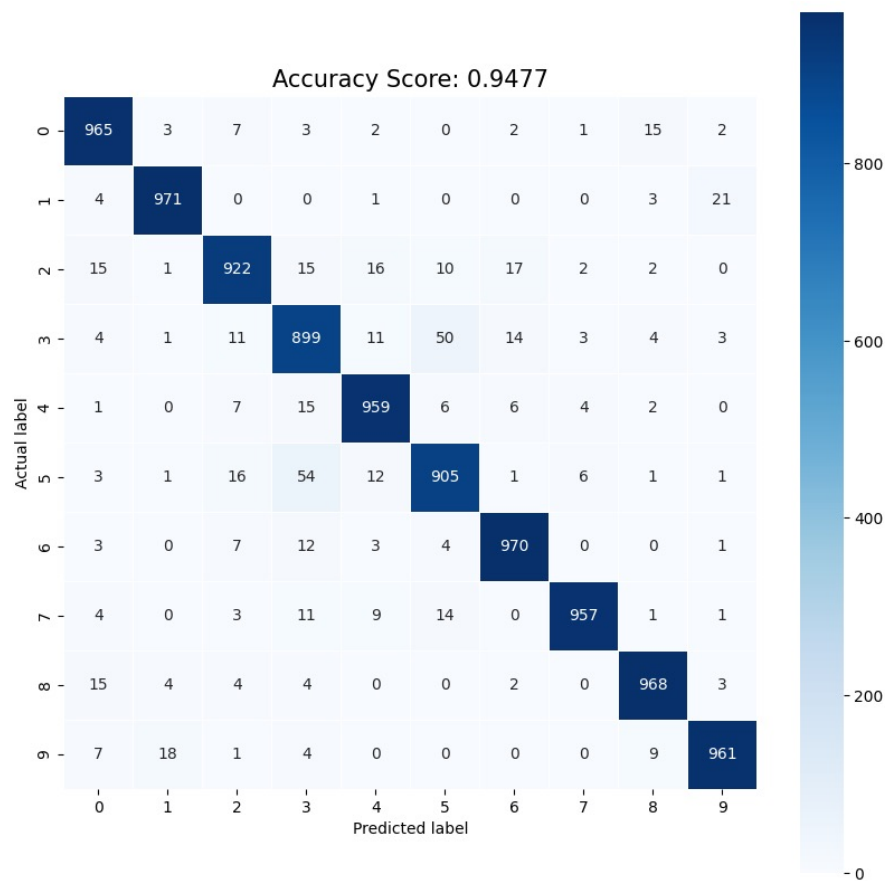


Figure 4.5: Confusion Matrix between Classification Results of Individual Classes on the CIFAR10 Dataset for the VGG-small Model with Full Precision

distinguish. Birds are similar to multiple classes due to their diverse postures and animal characteristics, resulting in the highest sum of Jaccard Similarity with other classes. This suggests that Jaccard Similarity correlates with classification task difficulty.

Table 4.4 lists the ten most confusing class pairs, showing a correlation between path overlap and categorization difficulty. The greater the critical path overlap between two classes, the more challenging their categorization task, leading to more confusion.

Having discovered the correlation between classification difficulty and path overlap, we need to explore its application to the quantization process. Specifically, we want to see if reducing computational accuracy impacts harder classification tasks more than easier ones. Intuitively, classes with high path overlap will produce more confusion when computational precision is reduced, as lower quantization bits cannot precisely distinguish overlapping parts. To investigate this, we obtained the confusion matrices of VGG-small+CIFAR10 for different quantization bit counts, shown in Figure 4.7. We aim to determine if more difficult classifications generate more errors, given their higher accuracy requirements. From the full-precision model to the 3-bit quantized model, the performance dropped significantly, adding 4316 misclassified data points.

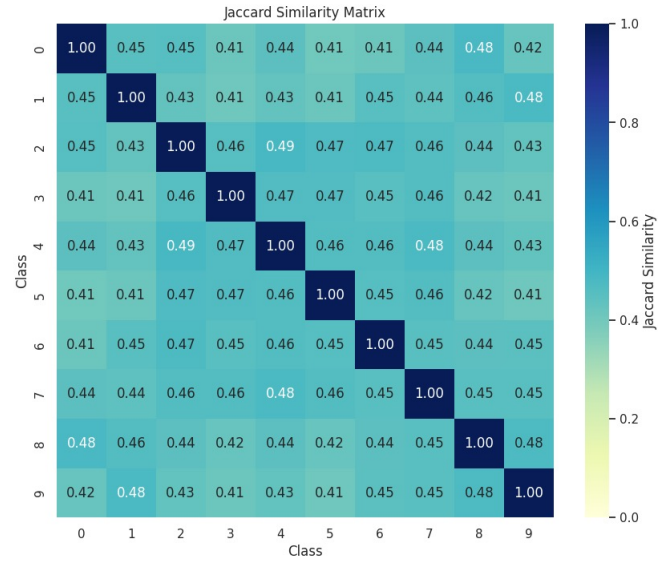
Classifying cats and dogs within CIFAR10 is particularly challenging, attributed to the high Jaccard Similarity (47%) of their critical paths in models like VGG-small and ResNet-20-x1. This overlap highlights the complexity and potential for misclassification. Comparing full-precision and 3-bit quantized models, confusion between cats and dogs increased, with mislabeled data points rising from 182 to 302.

Distinguishing cars from trucks in the CIFAR10 dataset is also challenging. These models, especially VGG-small, show a critical path Jaccard similarity of 48/

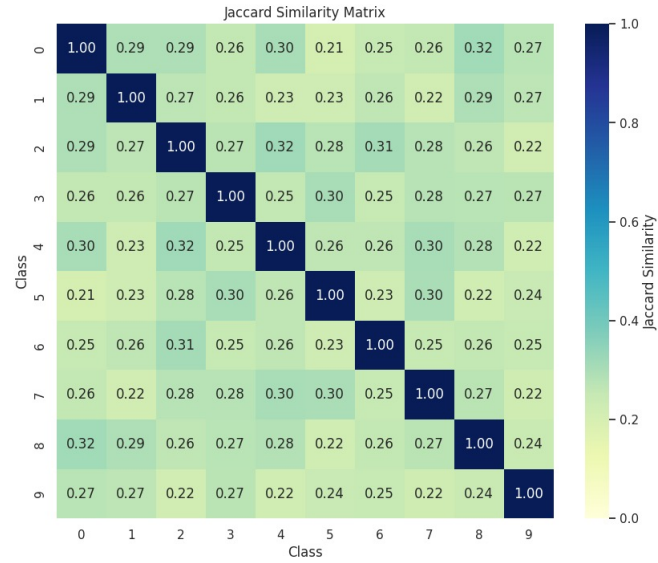
The bird class has the biggest sum of Jaccard Similarity with other classes, complicating accurate classification. The impact of quantization is evident: the model's accuracy for bird classification dropped from 63% to 35%, the steepest decline among all categories. The extent of confusion between birds and other classes positively correlates with the Jaccard Similarity of their critical paths.

Some tasks, intuitively simpler, show lower Jaccard Similarity. For example, misclassifications between airplanes and dogs in the VGG-small model have a Jaccard Similarity of 41%. During 3-bit quantization, confusion instances between airplanes and dogs increased from 28 to 65. This increase, though noticeable, is much smaller compared to the previously mentioned categories, highlighting that not all classification tasks are equally challenging.

In the earlier experiment, we drew two significant conclusions:



(a) Jaccard Similarity Matrix of VGG-small + CIFAR10



(b) Jaccard Similarity Matrix of ResNet-20-x1 + CIFAR10

Figure 4.6: Jaccard Similarity Matrix of VGG-small + CIFAR10 and ResNet-20-x1 + CIFAR10 Experiments

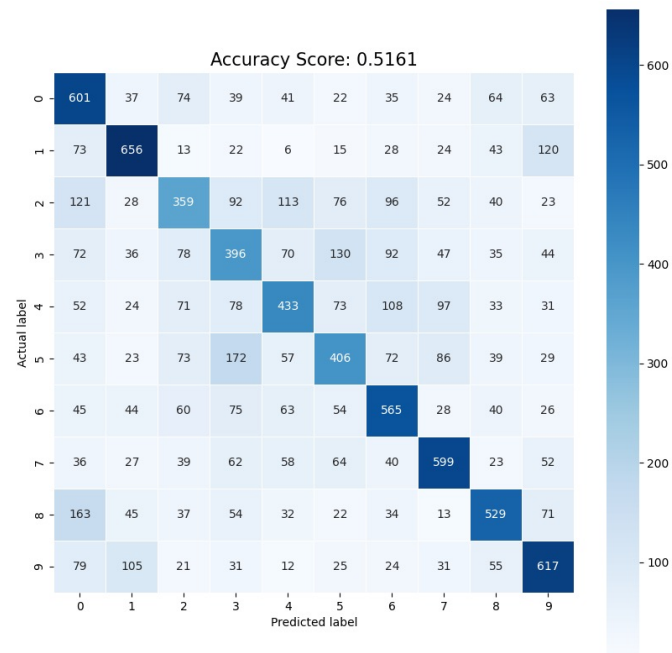


Figure 4.7: Confusion Matrix between Classification Results of Individual Classes on the CIFAR10 Dataset for the VGG-small Model with 3bit Quantization

First, the Jaccard Similarity of critical paths across various classes is crucial in understanding classification confusion. There is a strong positive correlation between the Jaccard Similarity of these paths and the level of confusion in categorization tasks. Higher Jaccard Similarity values between any two classes indicate a greater classification challenge. Similarly, a higher cumulative Jaccard Similarity for a class against all other classes indicates greater classification difficulty for that class.

Second, in the context of model quantization, characterized by reduced computational accuracy, we observed a distinct pattern: as computational accuracy diminishes, classification tasks with higher Jaccard Similarity values experience a more rapid decline in accuracy. This trend highlights the significant potential of Jaccard Similarity on improvement of model's performance under constrained computational conditions.

These conclusions provide theoretical support for our class-path-based quantization approach.

4.5 Class-path-based Quantization Experiment

We compared class-path-based quantization with class-based quantization under equal conditions. The result of class-based quantization were obtained from [48].

During the model training phase, we selected specific parameters to optimize performance and ensure effective learning. For ResNet models, the learning rate was initialized at 0.1, while VGG-small models started at 0.02, based on their unique architectures. The learning rate was reduced by a factor of 10 at the 100th, 150th, and 300th epochs to adapt to changes in the loss function and avoid local minima.

Momentum, set at 0.9 for all models, helped accelerate gradients in the right direction, ensuring stable and faster convergence. Regularization was achieved through weight decay, set at 0.0001 for ResNet and 0.0005 for VGG-small models to match their architectural needs.

The batch size was uniformly set at 100 across all datasets to balance computational efficiency and accuracy. To prevent overfitting and ensure generalization to unseen data, training was monitored and concluded after 400 epochs, determined to be optimal for learning intricate patterns without overfitting.

In our experiment, we adhered to the bit-width allocation for weights as described in Section 3.5. For activations, we directly assigned their target bit-widths without complex computation.

During the refining phase, we maintained continuity by using the same optimization parameters from the training phase. This consistency helps fine-tune the model’s weights and maintains the integrity of the learning process.

We did not quantize the first and output layers of our neural network models, following a strategy inspired by previous research [48].

To standardize our subsequent experiments, we introduced an average bit-width setting for the weights, represented as k . This value is the arithmetic average of all the bit-widths across the quantized weights. Mathematically, k is determined as:

$$k = \frac{\sum_{i=1}^N b_i}{N} \quad (4.1)$$

where N represents the totality of weights, excluding those from the first and the output layers. b_i corresponds to the bit-width of the i -th weight in the network.

In the refining phase, we introduced knowledge distillation loss to transfer knowledge from a larger model (teacher) to a smaller model (student), improving the student’s performance. The weighting factor α in equation 4.1 was fixed at 0.3 to balance the original loss with the distillation loss.

To explore the effects of different bit allocation strategies on model performance, we used the VGG-small+CIFAR10 setup. Classes were organized by sensitivity, and bit counts were manually assigned to each class to examine performance variations. To allow for comprehensive observations and analysis, we set the target average bit count to 4 bits.

For this experiment, we explored three bit allocation strategies: Compact Distribution, Bimodal Distribution, and Uniform Distribution. Within a search space of 1 to 6 bits, targeting an average of 4 bits, Compact Distribution clusters most bit-widths around 3, 4, and 5. Bimodal Distribution has bit-widths primarily at 1, 2, 5, and 6, while Uniform Distribution evenly spreads bit-widths across classes.

The experimental outcomes, shown in Table 4.5, indicate that the choice of bit allocation strategy does not significantly influence model performance. This suggests minimal need for exhaustive manual adjustments during the search process, highlighting the robustness of our proposed quantization strategy.

In a detailed exploration, we evaluated two strategies for extracting critical paths: one with a shared threshold and one without. Experimental comparisons showed that these configurations had minimal impact on overall model performance. This may be due to the relatively low complexity of the VGG-small+ CIFAR10 setup and the robustness of our quantization strategy.

However, we observed a critical issue: setting the sparsity too low must be avoided. When sparsity is too low, highly sensitive categories exert excessive influence. If the bit-widths for these categories exceed the target, the bit-widths for other categories must be set very low to maintain the desired average. In extreme cases, this resulted in most other categories having a bit count of 1. Even slightly exceeding the target for the most sensitive category made it difficult to achieve the overall average bit target. Manual tuning and optimization are necessary to ensure consistency with the expected performance baseline.

We compared the quantization approach based on critical paths (denoted as ClassPathQ) with the class-based quantization method (CQ). The comparison between the accuracy of Critical-path-based Quantization(ClassPathQ) and class-based Quantization(CQ) is shown in Figure 4.8. In the designated experimental setups, both methodologies were subjected to equivalent testing environments and conditions. Within the VGG-small+ CIFAR10 configuration, with a target average bit-width of 2.0 for both weights and activations, ClassPathQ conspicuously outperformed CQ. Nonetheless, for the configurations of 3.0\3.0 and 4.0\4.0, the performance difference between the two was small.

In the VGG-small+ CIFAR100 tests, the critical-path-based quantization strategy showed a significant advantage. This highlights ClassPathQ’s ability to handle complex classification tasks like CIFAR100, consistently delivering superior accuracy in the 2.0\2.0, 3.0\3.0, and 4.0\4.0 conditions.

Conversely, in the ResNet-20-x1+ CIFAR10 setup, ClassPathQ and CQ performed almost identically. The ResNet-20-x1 architecture has relatively few neurons or filters per layer and is quite deep, reflecting a ”narrow and deep” structure. In such architectures, the difference between critical-path-based quantization and class-based quantization may be small. Each critical path in this structure is highly sensitive, challenging mixed precision quantization.

In contrast, in the ResNet-20-x5+ CIFAR100 setup, with more neurons or filters per layer and increased classification complexity, ClassPathQ outperformed CQ, particularly in the 2.0\2.0 and 3.0\3.0 scenarios.

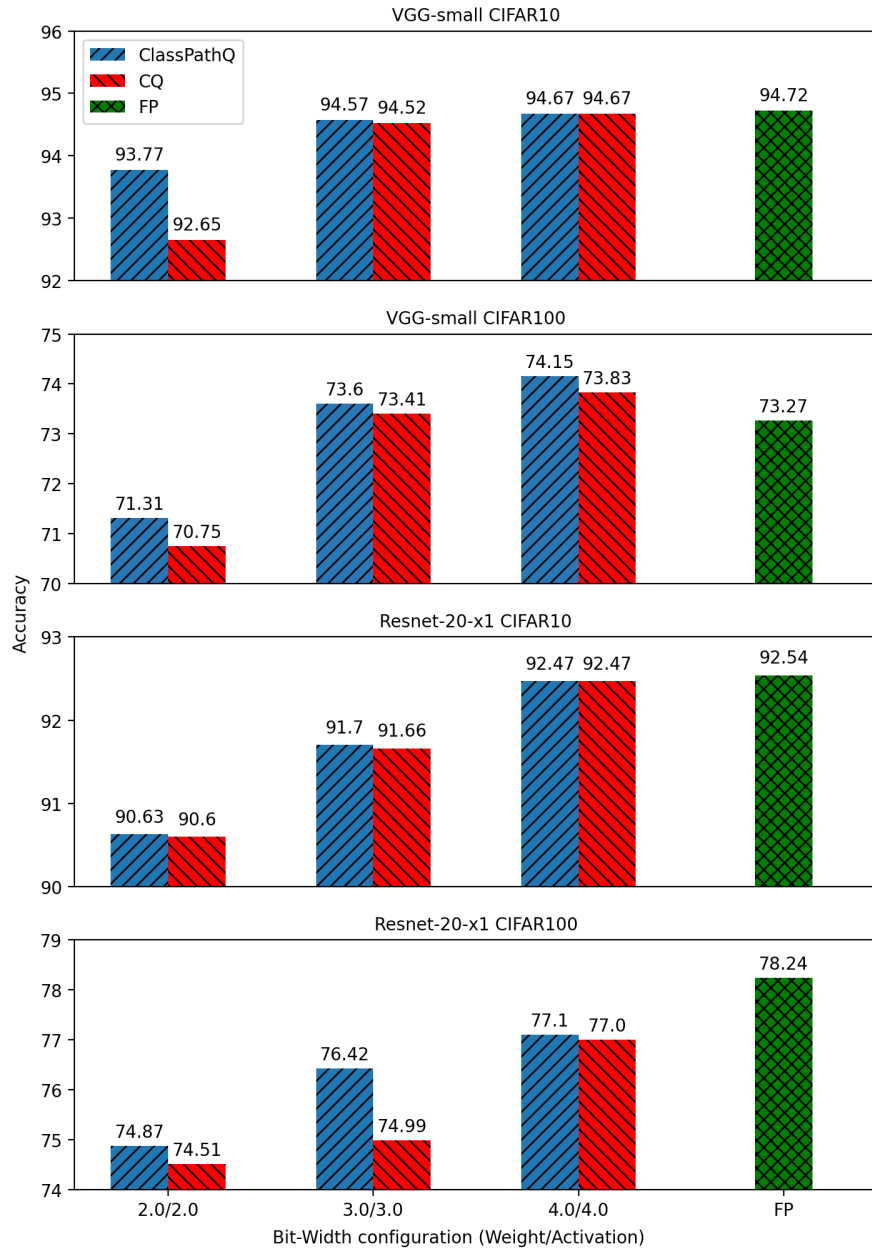


Figure 4.8: Comparison of accuracy between ClassPathQ and CQ[48] with 2.0\2.0, 3.0\3.0, and 4.0\4.0 bit-width settings and full-precision models. The blue bars are the proposed method, and the red bars are CQ. The green bars are the full-precision baseline models

Layer Type	Configuration
Input	$3 \times 32 \times 32$
Convolution	128 filters, 3×3 , stride 1, padding 1
Batch Normalization	128 features
ReLU	-
Convolution	128 filters, 3×3 , stride 1, padding 1
Batch Normalization	128 features
ReLU	-
Max Pooling	2×2 , stride 2
Convolution	256 filters, 3×3 , stride 1, padding 1
Batch Normalization	256 features
ReLU	-
Convolution	256 filters, 3×3 , stride 1, padding 1
Batch Normalization	256 features
ReLU	-
Max Pooling	2×2 , stride 2
Convolution	512 filters, 3×3 , stride 1, padding 1
Batch Normalization	512 features
ReLU	-
Convolution	512 filters, 3×3 , stride 1, padding 1
Batch Normalization	512 features
ReLU	-
Max Pooling	2×2 , stride 2
Fully Connected	$512 \times 4 \times 4$ to 1024 neurons
Batch Normalization	1024 features
ReLU	-
Fully Connected	1024 to 1024 neurons
Batch Normalization	1024 features
ReLU	-
Fully Connected	1024 to class_num

Table 4.1: Structure of the VGG-small model.

Layer Type	Configuration
Input	$3 \times 32 \times 32$
Convolution	3 to 16, 3×3 , stride 1, padding 1
Batch Normalization	16 features
ReLU	-
<i>First Set of Basic Blocks</i>	
BasicBlock1	2 Conv2d (16 filters), 2 BatchNorm, 2 ReLU
BasicBlock2	2 Conv2d (16 filters), 2 BatchNorm, 2 ReLU
BasicBlock3	2 Conv2d (16 filters), 2 BatchNorm, 2 ReLU
<i>Second Set of Basic Blocks</i>	
BasicBlock1	2 Conv2d (32 filters), 2 BatchNorm, 2 ReLU
BasicBlock2	2 Conv2d (32 filters), 2 BatchNorm, 2 ReLU
BasicBlock3	2 Conv2d (32 filters), 2 BatchNorm, 2 ReLU
<i>Third Set of Basic Blocks</i>	
BasicBlock1	2 Conv2d (64 filters), 2 BatchNorm, 2 ReLU
BasicBlock2	2 Conv2d (64 filters), 2 BatchNorm, 2 ReLU
BasicBlock3	2 Conv2d (64 filters), 2 BatchNorm, 2 ReLU
Average Pooling	Pool size: 4×4
Fully Connected	64 to num_classes

Table 4.2: Structure of the ResNet-20x1 model.

Removed Class Path	Class 0 Acc.	Class 1 Acc.	Class 2 Acc.	Class 3 Acc.	Class 4 Acc.	Class 5 Acc.	Class 6 Acc.	Class 7 Acc.	Class 8 Acc.	Class 9 Acc.
None	91.80%	93.10%	79.70%	73.60%	86.40%	83.00%	91.40%	90.80%	93.30%	92.80%
Class 0	91.50%	93.00%	79.40%	73.80%	86.20%	81.90%	90.70%	90.00%	92.90%	92.60%
Class 1	91.50%	93.00%	79.00%	72.40%	85.50%	82.90%	91.40%	90.30%	93.30%	92.90%
Class 2	91.70%	92.90%	79.10%	72.70%	85.30%	82.60%	91.20%	90.20%	93.30%	93.00%
Class 3	91.60%	93.20%	78.90%	72.50%	85.10%	82.40%	91.10%	90.10%	93.40%	92.80%
Class 4	91.60%	93.10%	79.20%	72.70%	86.00%	82.20%	91.40%	90.30%	93.10%	93.30%
Class 5	91.60%	93.10%	79.60%	72.90%	85.90%	82.50%	91.50%	90.50%	93.60%	93.00%
Class 6	91.70%	93.00%	79.20%	72.00%	85.90%	82.70%	91.50%	90.10%	93.40%	93.20%
Class 7	91.50%	93.00%	79.20%	73.00%	86.40%	83.20%	91.70%	90.00%	93.50%	93.10%
Class 8	91.50%	92.50%	79.60%	72.80%	86.10%	82.60%	90.80%	90.00%	93.10%	93.00%
Class 9	92.10%	93.20%	79.40%	72.70%	86.30%	82.60%	91.40%	90.70%	92.10%	92.10%
Unimportant Neuron	0.3%	2.7%	23.6%	30.0%	15.8%	32.7%	0%	0%	0.10%	3.7%

Table 4.3: Result of Removing Single Class Path Experiment

actual label	actual class	predicted label	predicted class	confusion count
3	Cat	5	Dog	92
5	Dog	3	Cat	90
2	Bird	4	Deer	48
2	Bird	3	Cat	42
2	Bird	0	Airplane	41
1	Car	9	Truck	41
3	Cat	6	Frog	40
9	Truck	1	Car	35

Table 4.4: the Ten Most Confusing Classes and their Count for FP model

Shared Threshold	Bit Distribution	Class Bits	Accuracy
No	Compact	[3,3,5,2,3,3,2,3,4,3,3]	94.62%
No	Bimodal	[2,4,5,1,1,2,1,1,5,1,2]	94.64%
No	Uniform	[3,3,5,1,2,3,1,2,4,2,3]	94.65%
Yes	Compact	[3,3,3,2,2,5,2,2,4,2,2]	94.61%
Yes	Bimodal	[2,4,2,1,1,5,1,1,4,1,3]	94.62%
Yes	Uniform	[3,3,3,1,2,5,1,2,4,2,3]	94.60%

Table 4.5: Critical-path-based Quantization results for different manual settings with sparsity = 0.5 and target bit-width = 4.0

5 Conclusion and Future Work

In this study, we introduce a novel neural network quantization method based on classification difficulty. This approach begins by using the importance scores of neurons and filters to build critical paths for each category in the trained model. These paths guide the allocation of bit assignments. We detail the implementation procedures and explore various configurations and refinements.

First, we use the Important Neuron Theory to assess the importance score of each neuron for different classes. Next, we assign importance scores to each filter in the convolutional layers based on their significance to different classes. Using these scores, we extract the critical paths for each category.

We then analyze the similarities and differences between these critical paths using Jaccard Similarity. This allows us to rank the sensitivity of each class, guiding the allocation of appropriate bit counts to the critical paths for precise quantization. After assigning bits, we fine-tune the quantized model to ensure it maintains high performance. This step is crucial to preserving the model’s accuracy post-quantization.

To validate our neural network quantization strategy, we explored the relationship between critical paths and classification difficulty. Through experimental validations, we found a correlation between critical paths and the complexity of classification tasks. Our findings indicate that higher Jaccard Similarity between critical paths of two categories makes distinguishing them more challenging. Additionally, during quantization, reduced computational precision more adversely affects performance on difficult classification tasks, while easier tasks see a moderate decline in accuracy. These results provide theoretical support for our critical-path-based quantization strategy.

In subsequent quantization experiments, our approach not only matched but sometimes exceeded the performance of current state-of-the-art methods in complex classification scenarios. Specifically, at the same target average bit rate, our quantized models demonstrated better inference accuracy.

In our future research, we plan to deepen and expand our current work by exploring several key directions. First, we aim to experiment with various neuron significance evaluation methods, such as those incorporating second-order derivative information as proposed in the HAWQ papers [12][11], to more accurately gauge neuron contributions to classification tasks. Second, we intend to conduct rigorous experiments on the Interference Degree to understand its role in neural network quantization. Third, we will investigate new techniques for evaluating the sensitivity of crucial paths to determine which paths require particular attention during quantization. Fourth, we seek to develop new methods for assessing the complexity of classification tasks, providing a more accurate evaluation of task intricacies. Lastly, drawing from these insights, we aim to formulate advanced neural network quantization strategies to enhance model precision and efficiency. Together, these efforts promise a more comprehensive understanding of neural network quantization, paving the way for more effective and accurate methodologies.

Bibliography

- [1] R. Altilio, A. Rosato, and M. Panella. “A nonuniform quantizer for hardware implementation of neural networks”. In: *European Conference on Circuit Theory and Design (ECCTD)*. 2017.
- [2] J. Ba and R. Caruana. “Do deep nets really need to be deep?”. In: *Advances in Neural Information Processing Systems 27: Annual Conference on Neural Information Processing Systems 2014*. Montreal, Quebec, Canada, Dec. 2014, pp. 2654–2662.
- [3] Christopher M. Bishop. *Neural Networks for Pattern Recognition*. Oxford University Press, 1995.
- [4] Alexey Bochkovskiy, Chien-Yao Wang, and Hong-Yuan Mark Liao. “YOLOv4: Optimal Speed and Accuracy of Object Detection”. In: *arXiv preprint arXiv:2004.10934* (2020).
- [5] Mariusz Bojarski et al. “Deep Learning for Autonomous Vehicles: Challenges and Open Questions”. In: *IEEE Signal Processing Magazine* 36.6 (2019), pp. 8–17.
- [6] Tom Brown et al. “Language models are few-shot learners”. In: *Advances in neural information processing systems* 33 (2020), pp. 1877–1901.
- [7] C. Bucilua, R. Caruana, and A. Niculescu-Mizil. “Model compression”. In: *Proceedings of the 12th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. KDD ’06. 2006, pp. 535–541.
- [8] Yunhe Choi, Mostafa El-Khamy, and Jungwon Lee. “Model Compression via Distillation and Quantization”. In: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. 2018, pp. 703–712.
- [9] Misha Denil et al. “Predicting parameters in deep learning”. In: *Advances in neural information processing systems* 26 (2013).
- [10] Emily L Denton et al. “Exploiting linear structure within convolutional networks for efficient evaluation”. In: *Advances in neural information processing systems* 27 (2014).

-
-
- [11] Zhen Dong et al. “HAWQ-V2: Hessian aware trace-weighted quantization of neural networks”. In: *Advances in neural information processing systems* 33 (2020), pp. 18518–18529.
 - [12] Zhen Dong et al. “HAWQ: Hessian aware quantization of neural networks with mixed-precision”. In: *Proceedings of the IEEE/CVF International Conference on Computer Vision*. 2019, pp. 293–302.
 - [13] Yiming Gan et al. “Ptolemy: Architecture support for robust deep learning”. In: *2020 53rd Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE. 2020, pp. 241–255.
 - [14] Amir Gholami et al. “A survey of quantization methods for efficient neural network inference”. In: *Low-Power Computer Vision*. Chapman and Hall/CRC, 2022, pp. 291–326.
 - [15] Xavier Glorot, Antoine Bordes, and Yoshua Bengio. “Deep sparse rectifier neural networks”. In: *Proceedings of the fourteenth international conference on artificial intelligence and statistics*. JMLR Workshop and Conference Proceedings. 2011, pp. 315–323.
 - [16] Yunchao Gong et al. “Compressing deep convolutional networks using vector quantization”. In: *arXiv preprint arXiv:1412.6115* (2014).
 - [17] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. MIT Press, 2016.
 - [18] Song Han, Huizi Mao, and William J Dally. “Deep Compression: Compressing Deep Neural Networks with Pruning, Trained Quantization and Huffman Coding”. In: *International Conference on Learning Representations (ICLR)* (2016).
 - [19] Stephen Hanson and Lorien Pratt. “Comparing biases for minimal network construction with back-propagation”. In: *Advances in neural information processing systems* 1 (1988).
 - [20] Babak Hassibi and David Stork. “Second order derivatives for network pruning: Optimal brain surgeon”. In: *Advances in neural information processing systems* 5 (1992).
 - [21] Simon Haykin. *Neural Networks: A Comprehensive Foundation*. Prentice Hall PTR, 1998.
 - [22] Kaiming He et al. “Deep residual learning for image recognition”. In: (2016).

-
-
- [23] G. E. Hinton, O. Vinyals, and J. Dean. “Distilling the knowledge in a neural network”. In: *CoRR* abs/1503.02531 (2015). URL: <http://arxiv.org/abs/1503.02531>.
- [24] Sepp Hochreiter and Jürgen Schmidhuber. “Long Short-Term Memory”. In: *Neural Computation* 9.8 (1997), pp. 1735–1780.
- [25] L. Hou and J. T. Kwok. “Loss-aware weight quantization of deep networks”. In: *International Conference on Learning Representations (ICLR)*. 2018.
- [26] Qijing Huang et al. “CodeNet: Efficient deployment of input-adaptive object detection on embedded FPGAs”. In: *The 2021 ACM/SIGDA International Symposium on Field-Programmable Gate Arrays*. 2021, pp. 206–216.
- [27] B. Ham J. Lee D. Kim. “Network quantization with element-wise gradient scaling”. In: (2021).
- [28] Benoit Jacob et al. “Quantization and training of neural networks for efficient integer-arithmetic-only inference”. In: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. 2018.
- [29] Ashkan Khakzar et al. “Neural response interpretation through the lens of critical pathways”. In: *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*. 2021, pp. 13528–13538.
- [30] Raghuraman Krishnamoorthi. “Quantizing deep convolutional networks for efficient inference: A whitepaper”. In: *arXiv preprint arXiv:1806.08342* (2018). URL: <http://arxiv.org/abs/1806.08342>.
- [31] A. Krizhevsky and G. et al. Hinton. *Learning multiple layers of features from tiny images*. Tech. rep. Tech Report, 2009.
- [32] Alex Krizhevsky, Ilya Sutskever, and Geoffrey E. Hinton. “ImageNet Classification with Deep Convolutional Neural Networks”. In: *Advances in Neural Information Processing Systems (NeurIPS)*. 2012, pp. 1097–1105.
- [33] Yann LeCun et al. “Advances in neural information processing systems”. In: *San Francisco, CA, USA: Morgan Kaufmann Publishers Inc* (1990), pp. 396–404.
- [34] Yann LeCun et al. “Gradient-Based Learning Applied to Document Recognition”. In: *Proceedings of the IEEE* 86.11 (1998), pp. 2278–2324.
- [35] Junghyup Lee, Dohyung Kim, and Bumsu Ham. “Network quantization with element-wise gradient scaling”. In: *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. June 2021, pp. 6448–6457.

-
-
- [36] Shiwen Mao and Zhenhua Chen. “Edge Computing and Artificial Intelligence: A Convergence”. In: *IEEE Internet of Things Journal* 5.6 (2017), pp. 4504–4512.
- [37] Paulius Micikevicius et al. “Mixed Precision Training”. In: *International Conference on Learning Representations (ICLR)*. 2018.
- [38] Renkun Ni et al. “WrapNet: Neural net inference with ultra-low-precision arithmetic”. In: (2021).
- [39] Yuxian Qiu et al. “Adversarial defense through network profiling based path extraction”. In: *IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 2019, pp. 4777–4786.
- [40] Z. Qu et al. “Adaptive loss-aware quantization for multi-bit networks”. In: *The IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 2020.
- [41] Alec Radford et al. “Improving language understanding by generative pre-training”. In: (2018).
- [42] Joseph Redmon and Ali Farhadi. *YOLOv3: An incremental improvement*. 2018. arXiv: 1804.02767 [cs.CV].
- [43] Joseph Redmon et al. “You Only Look Once: Unified, Real-Time Object Detection”. In: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. 2016, pp. 779–788.
- [44] David E Rumelhart, Geoffrey E Hinton, and Ronald J Williams. “Learning representations by back-propagating errors”. In: *nature* 323.6088 (1986), pp. 533–536.
- [45] Mark Sandler et al. “MobileNetV2: Inverted Residuals and Linear Bottlenecks”. In: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. 2018, pp. 4510–4520.
- [46] Vikas Sindhwani, Tara Sainath, and Sanjiv Kumar. “Structured transforms for small-footprint deep learning”. In: *Advances in Neural Information Processing Systems* 28 (2015).
- [47] Emma Strubell, Ananya Ganesh, and Andrew McCallum. “Energy and Policy Considerations for Deep Learning in NLP”. In: *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics (ACL)*. 2019, pp. 3645–3650.
- [48] Wenhao Sun et al. “Class-based quantization for neural networks”. In: *2023 Design, Automation and Test in Europe Conference and Exhibition (DATE)*. 2023, pp. 1–6.

-
-
- [49] Mukund Sundararajan, Ankur Taly, and Qiqi Yan. “Axiomatic attribution for deep networks”. In: *34th International Conference on Machine Learning, ICML 2017*. 2, 4, 7. 2017.
 - [50] Mingxing Tan and Quoc V Le. “EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks”. In: *Proceedings of the International Conference on Machine Learning (ICML)*. 2019, pp. 6105–6114.
 - [51] Kuan Wang et al. “HAQ: Hardware-aware automated quantization with mixed precision”. In: *IEEE/CVF conference on computer vision and pattern recognition*. 2019, pp. 8612–8620.
 - [52] Yulong Wang et al. “Interpret neural networks by identifying critical data routing paths”. In: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*. 1, 4, 5. 2018, pp. 8906–8914.
 - [53] Jason Yosinski et al. “Understanding Neural Networks through Deep Visualization”. In: *Deep Learning Workshop, International Conference on Machine Learning (ICML)*. 2015.
 - [54] Haichao Yu et al. “Any-precision deep neural networks”. In: *Association for the Advancement of Artificial Intelligence*. Vol. 35. 12. 2021, pp. 10763–10771.
 - [55] Sijie Zhao, Tao Yue, and Xuemei Hu. “Distribution-aware adaptive multi-bit quantization”. In: *IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 2021, pp. 9281–9290.
 - [56] A. Zhou et al. “Explicit loss-error-aware quantization for low-bit deep neural networks”. In: *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. 2018.